

Arquitectura de Integración Bancaria

Autor: Guillermo Alexis Barcia

Fecha: Julio del 2026

Campo	Valor
Objetivo	Diseñar una arquitectura de integración de alto nivel para modernización bancaria, con diagramas C4 claros y alineación al estándar BIAN.
Alcance	Core tradicional, core digital, banca web/móvil, pagos, Open Finance, riesgo, fraude, plataforma cloud, IAM, HA/DR, multicore, APIs, gobierno y migración.
Referencia cloud	Amazon Web Services (AWS) como nube principal. Equivalentes Azure documentados donde aplica.
Diagramas C4	Contexto y Contenedores al inicio (alto nivel); Componentes y Despliegue en sección final.

Resumen ejecutivo

Este documento presenta la propuesta de arquitectura de integración para la modernización de un banco tradicional. La solución central es una **Plataforma de Integración Bancaria** desplegada en nube, que actúa como capa de mediación entre canales digitales (web, móvil), terceros de Open Finance y los sistemas backend: core tradicional, core digital, pagos, riesgo y fraude.

La arquitectura resuelve el problema de convivencia entre sistemas legacy y digitales mediante un **Core Router Multicore** que enruta operaciones de forma transparente, sin que los canales conozcan en qué core reside cada producto. Durante la migración se aplica el patrón **Strangler Fig** para minimizar riesgo operativo.

Alineación al estándar BIAN (Banking Industry Architecture Network): el diseño arquitectónico organiza la capa de integración por **Service Domains BIAN** (Current Account, Payment Execution, Fraud Evaluation, Risk Management, Customer Offer, Open Banking) y no por sistema backend. Cada dominio de negocio bancario se implementa como un microservicio de integración con modelo canónico interno, adaptadores hacia sistemas externos y publicación de eventos de dominio. Esta alineación permite que la arquitectura evolucione por capacidad de negocio y sobreviva al reemplazo del core legacy sin rediseño de canales ni contratos de API.

Visión por capas de la arquitectura

- **Capa 1 - Canales:** Banca Web, Banca Móvil y consumidores Open Finance (terceros regulados).
- **Capa 2 - Perímetro y experiencia:** API Gateway + WAF, IAM (Cognito/Identity Center), BFF Web y BFF Móvil.
- **Capa 3 - Dominio de integración:** Microservicios alineados a BIAN, Core Router multicore, Orquestador Saga y Event Bus (Kafka/MSK).
- **Capa 4 - Sistemas backend:** Core tradicional (on-prem), core digital (nube), pagos, riesgo, fraude, AML y notificaciones.

Los diagramas **C4** están adjuntos en la **sección 13**.

Mapeo BIAN - Service Domains a microservicios

Service Domain BIAN	Microservicio de integración	Sistemas integrados
Current Account	account-integration-svc	Core legacy + core digital
Payment Execution	payment-integration-svc	Pagos, riesgo, fraude
Customer Offer	customer-integration-svc	Core digital, KYC/AML
Fraud Evaluation	fraud-integration-svc	Motor de fraude, event bus
Risk Management	risk-integration-svc	Motor de riesgos
Open Banking	API Gateway + IAM	Terceros Open Finance

Tabla de contenido

- Resumen ejecutivo - Arquitectura de alto nivel
- 1. Introducción y contexto del problema
- 2. Visión arquitectónica y alineación BIAN
- 3. Patrones de integración: decisiones fundamentadas
- 4. Propuesta de solución en la nube (AWS)
- 5. Gestión de identidad y acceso (IAM)
- 6. Seguridad, cumplimiento normativo y LOPD
- 7. Estrategia de integración multicore
- 8. Estrategia de APIs internas/externas y mensajería
- 9. Gobierno de APIs y microservicios
- 10. Plan de migración gradual
- 11. Modelo de diseño de observabilidad y trazabilidad
- 12. Registro de decisiones arquitectónicas (ADRs)
- 13. Diagramas C4
- 14. Conclusiones

1. Introducción y contexto del problema

Un banco en proceso de modernización enfrenta un problema de integración heterogénea: conviven sistemas con décadas de antigüedad (core tradicional con batch, colas propietarias y protocolos cerrados) y plataformas digitales cloud-native (core digital, canales web/móvil, motores de riesgo y fraude en tiempo real). Agregar integraciones punto a punto entre cada par de sistemas genera acoplamiento, deuda técnica y riesgo operativo insostenible.

La respuesta arquitectónica es una Plataforma de Integración Bancaria centralizada, desplegada en nube, que actúa como capa de mediación entre canales, terceros regulados (Open Finance) y backends. Esta plataforma no reemplaza los cores: los orquesta, enruta y sincroniza durante un período de convivencia que puede extenderse varios años (estrategia multicore con patrón Strangler Fig).

1.1 Drivers de negocio y técnicos

Cada driver de negocio se traduce en un requisito no funcional que condiciona el diseño. Ignorar esta trazabilidad produce arquitecturas genéricas que no resisten auditoría técnica.

Driver	Requisito derivado	Componente que lo resuelve
Experiencia digital competitiva	Latencia p95 < 500 ms en consultas	BFF, Redis, APIs optimizadas
Open Finance / regulación	Consentimiento, mTLS, auditoría	API Gateway, IAM, Audit Trail
Continuidad operativa	99.95% disponibilidad, RTO <= 30 min	EKS multiregión, MSK replicado, Route 53
Convivencia multicore	Enrutamiento transparente legacy/digital	Core Router Multicore
Protección de datos (LOPD)	Minimización PII, consentimiento, retención	Tokenización, políticas de logs, OpenSearch
Time-to-market	Despliegue semanal sin downtime	EKS, GitOps, feature flags

1.2 Supuestos y restricciones

- El core tradicional permanece en datacenter on-prem o nube privada con conectividad dedicada (AWS Direct Connect o Azure ExpressRoute).
- El banco ya cuenta o adoptará un IdP corporativo (Active Directory / Entra ID) para empleados.
- Los terceros Open Finance son entidades reguladas que presentan certificados emitidos por CA

autorizada.

- El volumen estimado es de 5.000-15.000 TPS en picos (fin de mes) para canales digitales.
- El equipo de plataforma tiene experiencia en Kubernetes; no se adopta serverless puro para evitar cold starts en pagos.

2. Visión arquitectónica y alineación BIAN

La arquitectura se estructura en cuatro capas lógicas, visibles en el diagrama C4 de Contenedores: (1) Canales, (2) Perímetro y experiencia, (3) Dominio de integración, (4) Sistemas backend. Esta separación sigue el principio de Separation of Concerns: cada capa evoluciona con ritmo independiente.

La organización del dominio de integración sigue el estándar BIAN (Banking Industry Architecture Network). BIAN define Service Domains que representan capacidades de negocio bancario (cuentas corrientes, ejecución de pagos, evaluación de fraude, etc.). Mapear microservicios a dominios BIAN — y no a sistemas backend — permite que la arquitectura sobreviva al reemplazo del core legacy sin rediseñar la capa de canales.

Service Domain BIAN	Microservicio	Responsabilidad	Sistemas integrados
Current Account	account-integration-svc	Cuentas, saldos, movimientos	Core legacy, core digital
Payment Execution	payment-integration-svc	Transferencias, pagos, ordenes	Pagos, riesgo, fraude
Customer Offer	customer-integration-svc	Alta cliente, productos	Core digital, KYC/AML
Fraud Evaluation	fraud-integration-svc	Señales y alertas antifraude	Motor fraude, Kafka
Risk Management	risk-integration-svc	Scoring y límites	Motor riesgos
Open Banking	API Gateway + políticas IAM	Exposición regulada a terceros	Terceros Open Finance

ADR-001: Organización por dominio BIAN

Contexto: El banco tiene 6+ sistemas backend con equipos y ciclos de vida distintos.

Decisión: Crear microservicios de integración alineados a Service Domains BIAN, no a sistemas backend.

Justificación: Cuando el core legacy se retire, solo cambian los adaptadores del dominio afectado; los canales y contratos de API permanecen estables. Reduce el blast radius de cambios.

Alternativas descartadas: Integración por sistema (un ESB monolítico): descartado por acoplamiento y cuello de botella. Integración punto a punto: descartado por complejidad $O(n^2)$.

Referencia C4: Contenedores: Microservicios de Integración

ADR-002: Modelo canónico interno

Contexto: Core legacy representa una cuenta como registro COBOL; core digital como recurso REST JSON con estructura distinta.

Decisión: Definir un modelo canónico BIAN-aligned dentro de cada microservicio, con mappers hacia representaciones externas.

Justificación: La lógica de negocio opera sobre un único modelo; los adaptadores traducen al entrar/salir. Facilita agregación multicore (sumar saldos de ambos cores en una sola respuesta).

Alternativas descartadas: Transformación en ESB centralizado: descartado por convertir el bus en cuello de botella y punto único de fallo.

Referencia C4: Componentes: Canonical Mapper

3. Patrones de integración: decisiones fundamentadas

Cada patrón resuelve un problema específico de integración bancaria. A continuación se documenta el problema, la decisión, la ubicación en C4, la tecnología seleccionada y las alternativas descartadas.

ADR-003: API Gateway como único punto de entrada

Contexto: Canales digitales, terceros Open Finance y backoffice necesitan acceder a capacidades bancarias con políticas de seguridad homogéneas.

Decisión: Implementar Amazon API Gateway (HTTP API para internos, REST API para externos) con AWS WAF delante.

Justificación: Centraliza TLS terminación, validación JWT, rate limiting por client_id, versionado de rutas (/v1/, /v2/), logging a CloudWatch y throttling. Un único punto para aplicar parches de seguridad.

Alternativas descartadas: Kong self-hosted en EKS: mayor control pero más operación. Exposición directa de pods: inaceptable en sector regulado.

Referencia C4: Contenedores: API Gateway + WAF

ADR-004: Backend for Frontend (BFF) por canal

Contexto: La app móvil requiere payloads mínimos (menor consumo de datos); la web puede mostrar dashboards con datos agregados de múltiples dominios.

Decisión: Desplegar BFF Web y BFF Móvil como servicios independientes en EKS.

Justificación: Evita que el dominio de integración conozca detalles de UI. El BFF Móvil puede combinar 3 llamadas internas en 1 respuesta, reduciendo round-trips de 300ms a 100ms en redes móviles.

Alternativas descartadas: BFF único: descartado porque compromete optimización por canal. Sin BFF (canal llama directo a microservicios): expone complejidad y aumenta latencia.

Referencia C4: Contenedores: BFF Web, BFF Móvil

ADR-005: Arquitectura hexagonal con adaptadores

Contexto: Core legacy expone IBM MQ y archivos batch; core digital expone REST/gRPC; pagos usa ISO 20022 sobre API propietaria.

Decisión: Implementar puertos (interfaces) y adaptadores por protocolo en la Capa de Adaptadores.

Justificación: Un cambio en el protocolo del legacy (ej. migrar de MQ a Kafka) solo afecta el adaptador, no

la lógica de negocio. Facilita testing con adaptadores mock.

Alternativas descartadas: ESB con transformaciones XSLT centralizadas: descartado por dificultad de versionado y testing.

Referencia C4: Contenedores: Capa de Adaptadores; Componentes: Command/Query Adapters

ADR-006: Saga Orchestration para procesos distribuidos

Contexto: Una transferencia requiere: debitar cuenta, evaluar riesgo, verificar fraude, ejecutar pago y notificar. No existe transacción ACID entre 4 sistemas heterogéneos.

Decisión: Implementar orquestación tipo Saga con AWS Step Functions para procesos críticos y máquina de estados interna para procesos simples.

Justificación: Step Functions provee estado durable, reintentos configurables, compensaciones (reverso si pago falla tras debito) y visualización del flujo. Cumple auditoría al registrar cada paso.

Alternativas descartadas: 2PC (two-phase commit): descartado porque cores legacy no lo soportan. Coreografía pura (solo eventos): descartada para pagos por dificultad de trazabilidad y compensación.

Referencia C4: Contenedores: Orquestador Saga; Componentes: Saga Orchestrator

ADR-007: Arquitectura orientada a eventos

Contexto: El motor de fraude necesita analizar transacciones sin bloquear la respuesta al cliente. Las notificaciones (SMS/push) son inherentemente asíncronas.

Decisión: Implementar bus de eventos con Amazon MSK (Kafka gestionado), Schema Registry y topics por dominio.

Justificación: Kafka provee retención configurable (7-30 días), orden por partición (clave = account_id), throughput > 100K msg/s y replicación interregión con MirrorMaker 2. Desacopla productores de consumidores.

Alternativas descartadas: RabbitMQ: descartado para volumen bancario por limitaciones de throughput. SQS: descartado por no garantizar orden ni replay.

Referencia C4: Contenedores: Event Bus; Componentes: Event Publisher/Consumer

ADR-008: Idempotencia y Outbox Pattern

Contexto: En pagos, un timeout de red puede provocar reintentos del cliente o del gateway, generando cargos duplicados.

Decisión: Validar Idempotency-Key en Redis (TTL 24h) antes de ejecutar. Usar Outbox Pattern: escribir

evento en tabla outbox dentro de la misma transacción de negocio; un relay publica a Kafka.

Justificación: Garantiza exactamente-una-vez en efecto (at-least-once en transporte + idempotencia en consumidor). El Outbox evita inconsistencia entre BD y Kafka.

Alternativas descartadas: Confiar solo en reintentos del cliente: inaceptable en pagos. Distributed transactions (XA): no soportado entre Kafka y PostgreSQL de forma nativa.

Referencia C4: Componentes: Idempotency Manager, Event Publisher

4. Propuesta de solución en la nube (AWS)

La plataforma de integración se despliega como solución cloud-native en Amazon Web Services. La elección de AWS se fundamenta en: madurez de servicios gestionados para Kafka (MSK), Kubernetes (EKS), identidad (Cognito), observabilidad (CloudWatch/X-Ray) y conectividad híbrida (Direct Connect). Azure es viable como alternativa con mapeo directo (AKS, Event Hubs, API Management, Entra ID).

El principio rector es minimizar operación de infraestructura no diferenciadora: usar servicios gestionados para mensajería, identidad, secretos y observabilidad; reservar ingeniería para lógica de integración y adaptadores.

4.1 Topología de red y segmentación

La VPC de integración se despliega en dos regiones (us-east-1 primaria, us-west-2 DR) con topología idéntica. Cada VPC usa 3 Availability Zones para tolerancia a fallos de zona.

- **Subredes públicas:** solo Application Load Balancer interno y NAT Gateway. API Gateway es servicio gestionado sin subred dedicada.
- **Subredes privadas (app):** nodos EKS para microservicios, BFF y adaptadores.
- **Subredes privadas (data):** MSK brokers, ElastiCache Redis, OpenSearch.
- **Security Groups:** reglas restrictivas por tier (app solo habla con data en puertos específicos).
- **Network Policies (Kubernetes):** solo el namespace payment puede hablar con el adaptador de pagos.
- **Conectividad on-prem:** AWS Direct Connect (10 Gbps) + VPN IPSec como backup hacia datacenter del core legacy.
- **Service Mesh (opcional fase 2):** Istio o AWS App Mesh para mTLS entre pods y políticas de tráfico.

4.2 Mapeo detallado componente logico → servicio AWS

Componente C4	Servicio AWS	Configuración clave	Justificación
API Gateway + WAF	API Gateway REST + AWS WAF	Throttling 10K rps, WAF OWASP Top 10	Gestionado, integración nativa con Cognito y CloudWatch
BFF + Microservicios	Amazon EKS 1.29	3-20 nodos m5.xlarge, HPA, PDB	Portabilidad, GitOps, ecosistema Kubernetes
Event Bus	Amazon MSK	3 brokers kafka.m5.large, RF=3	Throughput, retención, MirrorMaker para DR

Componente C4	Servicio AWS	Configuración clave	Justificación
Schema Registry	AWS Glue Schema Registry	Compatibilidad BACKWARD	Validación de contratos sin operar Confluent
Cache / Idempotencia	ElastiCache Redis 7	Cluster mode, 3 shards, Multi-AZ	Sub-ms latency, TTL para idempotency keys
IAM clientes	Amazon Cognito	User Pool + MFA TOTP/SMS	OAuth2/OIDC nativo, integración móvil SDK
IAM empleados	IAM Identity Center	SSO SAML con AD corporativo	Federación sin replicar usuarios
Secretos	Secrets Manager + KMS	Rotación automática 90 días	Auditoría CloudTrail, cifrado CMK
Orquestador Saga	AWS Step Functions	Standard workflows, 90 días historial	Estado durable, compensaciones, visualización
Auditoría	Amazon OpenSearch	3 nodos r6g.large, ISM 7 años	Busqueda full-text para compliance
Observabilidad	CloudWatch + X-Ray + ADOT	Métricas, logs, trazas OTel	Correlación transaction-id end-to-end
CDN Web	CloudFront + S3	SPA estatica, TLS ACM	Latencia global < 50ms para assets
CI/CD	CodePipeline + ArgoCD	GitOps, scan Trivy en pipeline	Despliegue continuo con gates de seguridad
DR / DNS	Route 53 + health checks	Failover activo-pasivo entre regiones	RTO < 30 min automatizado
Conectividad legacy	Direct Connect + VPN	BGP, encriptación IPSec backup	Latencia predecible vs internet pública

4.3 Estrategia de despliegue y GitOps

Toda la infraestructura se define como código (Terraform) y los workloads como manifiestos Kubernetes

(Helm/Kustomize). ArgoCD sincroniza el estado deseado desde Git hacia ambos clusters EKS (primario y DR). Un cambio pasa por: PR → plan Terraform → merge → apply → ArgoCD sync → smoke tests → canary (Flagger) → promoción.

- Namespaces por dominio BIAN: ns-accounts, ns-payments, ns-fraud, ns-platform.
- Resource quotas y LimitRanges por namespace para evitar noisy neighbor.
- Pod Disruption Budgets: mínimo 2 replicas disponibles durante rolling updates.
- Horizontal Pod Autoscaler: escala por CPU (70%) y custom metric (requests/segundo).
- Imágenes en ECR con escaneo de vulnerabilidades y firma Cosign.

4.4 Alta disponibilidad y DR en nube

La topología activo-activo/activo-pasivo se detalla en el diagrama C4 de Despliegue. La región primaria (us-east-1) atiende el 100% del tráfico en condiciones normales. La región DR (us-west-2) mantiene EKS pods en standby (replicas mínimas), MSK replicado via MirrorMaker 2 y Redis con replica de lectura. Route 53 health check monitorea endpoint /health del API Gateway; si falla 3 veces consecutivas, cambia el registro DNS al API Gateway de DR.

Componente	HA en región	DR interregión	RPO	RTO
API Gateway	Regiónal (multi-AZ implícito)	Failover Route 53	0	5 min
EKS workloads	3 AZ, HPA, PDB	Cluster DR pre-desplegado	0 (stateless)	15 min
MSK Kafka	RF=3, min.insync=2	MirrorMaker 2	5 min	20 min
ElastiCache	Cluster Multi-AZ	Snapshot + restore manual	15 min	30 min
Cognito	Regiónal SLA AWS	Pool DR pre-configurado	0	30 min
OpenSearch	3 nodos Multi-AZ	Snapshot S3 cross-región	1 hora	1 hora

4.5 Estimación de capacidad (referencia)

Para 10.000 TPS pico con ratio 70% lectura / 30% escritura: EKS con 12-15 nodos m5.xlarge, MSK con 3 brokers kafka.m5.2xlarge, Redis cluster con 3 shards de 13GB. Estos valores se validan con pruebas de carga (Gatling/k6) antes de producción. FinOps: uso de Savings Plans para EKS y instancias reservadas para MSK; auto-scaling down en horario nocturno.

4.6 Equivalencias Azure (referencia)

AWS	Azure equivalente
EKS	AKS
MSK (Kafka)	Event Hubs (Kafka API)
API Gateway + WAF	API Management + WAF
Cognito	Entra ID B2C
IAM Identity Center	Entra ID SSO
ElastiCache Redis	Azure Cache for Redis
Secrets Manager	Key Vault
OpenSearch	Azure AI Search / Elastic Cloud
Direct Connect	ExpressRoute
Step Functions	Logic Apps / Durable Functions

ADR-009: Nube pública AWS como plataforma principal

Contexto: El banco necesita elasticidad, servicios gestionados y DR multiregión con time-to-market corto.

Decisión: Desplegar la plataforma de integración en AWS con topología híbrida (Direct Connect a on-prem).

Justificación: MSK y EKS son servicios maduros con SLA claros. El ecosistema de partners bancarios en AWS facilita integraciones. Direct Connect provee latencia < 5ms hacia datacenter.

Alternativas descartadas: On-prem puro: descartado por costo de operar Kafka y K8s. Multicloud activo-activo: descartado por complejidad operativa sin beneficio claro en fase inicial.

Referencia C4: Despliegue: 04-despliegue-nube

5. Gestión de identidad y acceso (IAM)

La gestión de identidad y acceso es transversal a toda la arquitectura. No es un componente aislado: cada request — ya sea de un cliente móvil, un tercero Open Finance o un microservicio interno — pasa por al menos dos capas de validación de identidad y autorización antes de ejecutar lógica de negocio.

El diseño IAM sigue el principio de Defense in Depth (defensa en profundidad) con tres capas: perímetro (API Gateway), servicio (Policy Enforcement) y dominio (reglas de negocio). Este enfoque se refleja en el diagrama C4 de Contenedores (contenedor IAM/IdP) y Componentes (Policy Enforcement).

5.1 Modelo de identidad por tipo de actor

Actor	IdP	Protocolo AuthN	AuthZ	Token	C4
Cliente banca web	Cognito User Pool	OIDC Auth Code + PKCE	Scopes OAuth2	JWT access_token 15 min	IAM → API GW → BFF Web
Cliente banca móvil	Cognito User Pool	OIDC Auth Code + PKCE + MFA	Scopes + device binding	JWT + refresh 30 días	IAM → API GW → BFF Móvil
Empleado backoffice	IAM Identity Center	SAML 2.0 SSO + MFA hardware	RBAC por rol	JWT SAML assertion	IAM → API GW
Tercero Open Finance	Cognito + CA propia	mTLS + Client Credentials	Scopes + consent_id	JWT 5 min, sin refresh	API GW → IAM
Microservicio EKS	IRSA	IAM Role (no usuario)	IAM Policy por servicio	Credenciales STS temporales	EKS Service Account
Adaptador → legacy	Secrets Manager	mTLS + API Key rotada	Credencial por adaptador	Cert X.509 + API Key	Adaptadores → VPN

5.2 Flujo detallado: cliente móvil (login + operación)

Paso 1 - Registro/Login: La app móvil redirige al Hosted UI de Cognito (o usa Amplify SDK). El usuario ingresa credenciales. Cognito valida y, si MFA está habilitado, solicita TOTP (Google Authenticator) o SMS OTP.

Paso 2 - Obtención de tokens: Cognito devuelve authorization_code. La app intercambia el code por access_token (JWT, 15 min), id_token (claims del usuario) y refresh_token (30 días) usando PKCE (code_verifier) para prevenir ataques de intercepción.

Paso 3 - Request autenticado: La app envía GET /v1/accounts con header Authorization: Bearer {access_token}. API Gateway valida: firma JWT (JWKS de Cognito), expiración, audience (client_id), issuer.

Paso 4 - Autorización en gateway: API Gateway verifica que el scope accounts:read este en el token. Si falta, responde 403. Si pasa, inyecta headers X-Customer-Id y X-Scopes al BFF.

Paso 5 - BFF y microservicio: BFF Móvil agrega datos de sesión y llama a account-integration-svc. Policy Enforcement valida ABAC: el customer_id del token debe coincidir con el account_id solicitado (un cliente no ve cuentas ajenas).

Paso 6 - Auditoría: Cada paso registra en Audit Trail: user_id, action, resource, timestamp, IP, device_id.

5.3 Flujo detallado: tercero Open Finance

Paso 1 - Registro del tercero: La fintech se registra en el portal de desarrolladores. Se genera client_id, se emite certificado mTLS (CA del banco o Let's Encrypt con validación empresarial) y se definen scopes permitidos (accounts:read, payments:initiate).

Paso 2 - Consentimiento del titular: El titular autoriza via app bancaria (OIDC consent screen). Se genera consent_id vinculado a client_id + customer_id + scopes + vigencia (max 90 días).

Paso 3 - Obtención de token: El tercero presenta certificado mTLS en TLS handshake. API Gateway valida contra truststore. El tercero solicita token con grant_type=client_credentials + consent_id. Cognito emite JWT de 5 minutos con scopes limitados al consentimiento.

Paso 4 - Consumo de API: Cada request incluye JWT + header X-Consent-Id. Policy Enforcement valida: scope suficiente, consentimiento vigente, tercero autorizado para ese cliente. Rate limit: 100 req/min por client_id (configurable por tier).

Paso 5 - Revocación: Si el titular revoca consentimiento, el consent_id se invalida en Redis. Requests posteriores reciben 403 con código consent_revoked.

5.4 Flujo: microservicio a microservicio (IRSA)

Los pods en EKS no usan credenciales estaticas. Cada Deployment tiene un ServiceAccount anotado con eks.amazonaws.com/role-arn que mapea a un IAM Role con políticas mínimas. Ejemplo: payment-integration-svc puede invocar solo el adaptador de pagos y publicar en topic banca.pagos.*. No puede leer secretos de otros namespaces ni acceder a MSK topics de fraude.

Las llamadas REST entre microservicios propagan el JWT original (forward) o usan token de servicio (client_credentials con scope interno) según política. La opción forward preserva el contexto del usuario para auditoría; la opción service-token se usa para procesos batch sin usuario.

5.5 Gestión del ciclo de vida de credenciales

- **Rotación de secrets:** Secrets Manager rota API Keys de legacy cada 90 días automáticamente. Los adaptadores leen la versión latest.
- **Certificados mTLS:** Monitoreo con alerta 30 días antes de vencimiento. Renovación automatizada via ACM Private CA.
- **Revocación de tokens:** Lista de revocación en Redis (blacklist por jti). Cognito admin API para invalidar refresh tokens comprometidos.
- **Accesos privilegiados:** Just-in-time access (JIT) para operaciones de backoffice. Sesión max 8 horas. Log enviado a SIEM.
- **Principio de mínimo privilegio:** Matriz de permisos revisada trimestralmente. Roles RBAC: customer, partner, compliance_reader, fraud_admin, platform_admin.

5.6 Autorización en tres capas (defense in depth)

Capa	Componente	Qué valida	Ejemplo de rechazo
1 - Perímetro	API Gateway + WAF	JWT valido, scopes, rate limit, IP	403 scope_insufficient
2 - Servicio	Policy Enforcement	ABAC: claims vs recurso	403 not_account_owner
3 - Dominio	Lógica de negocio	Reglas: montos, horarios, límites	422 daily_limit_exceeded

ADR-010: Cognito como IdP de clientes y terceros

Contexto: Se necesita OAuth2/OIDC nativo para web, móvil y Open Finance con MFA y escalabilidad.

Decisión: Amazon Cognito User Pools para clientes; Resource Server con scopes custom para terceros.

Justificación: Integración nativa con API Gateway (authorizer), SDK móvil (Amplify), MFA TOTP/SMS y SLA gestionado. Evita operar Keycloak en EKS.

Alternativas descartadas: Keycloak self-hosted: más flexible pero requiere operación HA. Auth0: costo elevado a escala bancaria.

Referencia C4: Contenedores: IAM/IdP

ADR-011: Autorización ABAC en Policy Enforcement

Contexto: RBAC puro (rol admin/user) es insuficiente: un cliente solo debe ver SUS cuentas; un tercero solo datos CON consentimiento.

Decisión: Implementar ABAC evaluando claims del JWT (customer_id, partner_id, scopes) contra el recurso solicitado.

Justificación: Granularidad necesaria para Open Finance y LOPD. El componente Policy Enforcement centraliza la lógica sin duplicarla en cada microservicio.

Alternativas descartadas: Autorización solo en API Gateway: insuficiente para reglas de negocio. OPA sidecar: viable en fase 2 para políticas más complejas.

Referencia C4: Componentes: Policy Enforcement

6. Seguridad, cumplimiento normativo y LOPD

Este apartado responde al requisito 3 del ejercicio: abordar seguridad, cumplimiento normativo y ley orgánica de protección de datos personales (LOPD). La propuesta no trata la seguridad como un anexo, sino como propiedad transversal del diseño: cada capa de la arquitectura incorpora controles verificables y trazables a componentes C4.

La argumentación se estructura en cuatro bloques: (1) marco de elementos de seguridad y privacidad, (2) protección de datos personales según LOPD, (3) cumplimiento normativo del sector bancario y Open Finance, (4) segregación de funciones y auditoría para regulador. La sección 5 (IAM) complementa este apartado con los mecanismos de autenticación y autorización.

6.0 Marco de elementos de seguridad, privacidad y cumplimiento

Los elementos que conforman este apartado se organizan en ocho pilares. Cada pilar se implementa en componentes concretos de la solución y se audita de forma continua.

Elemento	Qué garantiza	Cómo se implementa en la solución	Componente C4
Confidencialidad	Solo actores autorizados acceden a datos	TLS, mTLS, cifrado KMS, ABAC	API Gateway, IAM, Policy Enforcement
Integridad	Los datos no se alteran sin autorización	Firmas JWT, hashes, auditoría inmutable	Audit Trail, OpenSearch
Disponibilidad	Servicios accesibles ante fallos	HA multiregión, HPA, DR	EKS, MSK, Route 53
Autenticación	Verificar identidad del actor	Cognito, MFA, mTLS, SSO SAML	IAM / IdP
Autorización	Permisos según rol, scope y contexto	RBAC + ABAC en 3 capas	API Gateway, Policy Enforcement
Auditoría y trazabilidad	Evidencia para regulador e investigaciones	Logs correlacionados, transaction-id	Audit Trail, OpenSearch, CloudWatch
Privacidad (LOPD)	Derechos del titular y	Consentimiento,	Policy Enforcement,

Elemento	Qué garantiza	Cómo se implementa en la solución	Componente C4
	minimización PII	tokenización, borrado	customer-integration-svc
Cumplimiento normativo	Adherencia a regulación bancaria	Open Finance, KYC/AML, reportes	API Gateway, adaptadores AML

6.1 Protección de datos personales (LOPD)

La ley orgánica de protección de datos personales exige que el tratamiento de datos sea lícito, transparente y limitado a la finalidad. La arquitectura implementa privacy by design: los datos personales se minimizan en origen (eventos sin PII), se protegen en tránsito y reposo, y solo se exponen a terceros con consentimiento explícito del titular.

El derecho de acceso se garantiza via APIs de consulta donde ABAC asegura que el cliente solo accede a sus propios datos. El derecho de rectificación se atiende mediante APIs de actualización que propagan cambios a ambos cores via adaptadores. El derecho de supresión (borrado) se orquesta desde customer-integration-svc hacia legacy y digital. La portabilidad se habilita con APIs Open Finance en formato estándar. La oposición al tratamiento se registra como flag en el perfil del cliente y bloquea publicación de eventos de marketing en Kafka.

Principio / derecho LOPD	Implementación técnica	Componente C4	Justificación
Licitud y consentimiento	consent_id obligatorio para terceros Open Finance	Policy Enforcement	Sin consentimiento vigente no hay acceso a datos del titular
Minimización de datos	Eventos Kafka con customer_token, no PII en texto plano	Event Publisher, Schema Registry	Reduce superficie de exposición en mensajería
Limitación de finalidad	Scopes OAuth2 por recurso (accounts:read, payments:write)	IAM, API Gateway	Tercero solo accede a lo consentido
Exactitud (rectificación)	API PUT /customers/{id} propaga cambios a cores	customer-integration-svc, Adaptadores	Datos actualizados en todas las fuentes
Limitación de	Retención logs 7 años;	OpenSearch, MSK	Políticas automáticas de

Principio / derecho LOPD	Implementación técnica	Componente C4	Justificación
conservación	eventos Kafka 30 días (ISM/MSK)		expiración
Integridad y confidencialidad	TLS 1.3, KMS AES-256, tokenización PAN	API Gateway, Secrets Manager, KMS	Cifrado en tránsito y reposo por defecto
Derecho de acceso	ABAC: customer_id del token = recurso solicitado	Policy Enforcement	Titular solo ve sus propios datos
Derecho de supresión	API DELETE orquesta borrado en legacy y digital	customer-integration-svc	Cumple derecho al olvido
Portabilidad	API Open Finance exportación estándar JSON	API Gateway	Formato interoperable para el titular
Oposición	Flag marketing_opt_out bloquea eventos no esenciales	customer-integration-svc, Event Bus	Respeto a decisión del titular

6.2 Cifrado, perímetro seguro y DevSecOps

La protección en profundidad aplica controles en cada capa de la red y del ciclo de vida del software. El perímetro (WAF + API Gateway) filtra tráfico malicioso antes de llegar a la plataforma. El pipeline CI/CD incorpora gates de seguridad que impiden desplegar código o imágenes vulnerables.

- **Tránsito:** TLS 1.3 en canales externos; mTLS entre API Gateway y terceros; IPSec en Direct Connect hacia legacy.
- **Reposo:** KMS CMK para EBS, S3, MSK, Secrets Manager y OpenSearch. Rotación automática de llaves.
- **En uso:** tokenización de PAN via HSM o servicio externo; prohibición de almacenar CVV (PCI-DSS).
- **Perímetro:** AWS WAF con reglas OWASP Top 10, geo-blocking, rate limiting por IP y client_id.
- **DevSecOps:** SAST (SonarQube) en cada PR; DAST semanal; Trivy escanea imágenes; Cosign firma contenedores antes de ECR.
- **Detección:** integración con SIEM (CloudWatch Logs → alertas); correlación de eventos de seguridad con transaction-id.

6.3 Cumplimiento normativo del sector bancario

El cumplimiento normativo trasciende la LOPD e incluye regulaciones del sector financiero. La arquitectura

incorpora controles para Open Finance, prevención de lavado de activos (AML/KYC), auditoría para el regulador y trazabilidad de operaciones.

- **Oficial de cumplimiento:** accede via SSO con rol compliance_reader; solo lectura sobre reportes y evidencias (diagrama C4 Contexto).
- **Regulador / auditor:** recibe reportes exportados desde OpenSearch y APIs de cumplimiento; sin acceso directo a sistemas de producción.
- **Analista de fraude:** rol fraud_admin con acceso a alertas y casos, pero sin permiso para modificar transacciones (segregación).

Requisito normativo	Control en la solución	Componente C4	Evidencia para auditoría
Open Finance (acceso APIs terceros)	OAuth2 + mTLS + consent_id + scopes regulados	API Gateway, IAM, Policy Enforcement	Logs: quien accedio, a que recurso, con que consentimiento
KYC / AML (conozca su cliente)	Validación identidad en alta; consulta listas AML antes de operar	customer-integration-svc, adaptador AML	Registro de decisión AML por operación
Trazabilidad de transacciones	transaction-id propagado en headers, logs, trazas y eventos	Audit Trail, CloudWatch, X-Ray	Reconstrucción end-to-end de cualquier operación
Reportes al regulador	APIs de cumplimiento para oficial de compliance	API Gateway, OpenSearch	Consultas sobre evidencias sin acceso a producción
No repudio	Logs inmutables con timestamp, actor, acción, recurso	Audit Trail, OpenSearch ISM	Evidencia que no puede alterarse post-facto
PCI-DSS (datos de tarjeta)	Tokenización PAN; nunca almacenar CVV; segmentación de red	Adaptadores pagos, Vault, VPC	Scope PCI reducido a adaptador de pagos

6.4 Segregación de funciones (SoD)

La segregación de funciones (Separation of Duties) impide que una sola persona concentre permisos incompatibles. La arquitectura implementa SoD mediante RBAC con roles mutuamente excluyentes y ABAC que restringe acciones según contexto.

Función	Rol IAM	Puede hacer	No puede hacer
Operación bancaria (cliente)	customer	Ver y operar sus cuentas	Acceder a datos de otros clientes
Análisis de fraude	fraud_admin	Ver alertas, bloquear tarjetas	Modificar saldos ni aprobar transacciones
Cumplimiento	compliance_reader	Consultar reportes y evidencias	Ejecutar transacciones ni modificar datos
Administración plataforma	platform_admin	Gestionar infraestructura EKS	Acceder a datos de clientes (PII)
Tercero Open Finance	partner (scope limitado)	Datos consentidos del titular	Acceso sin consent_id vigente

6.5 Tabla resumen: requisito de seguridad/cumplimiento → solución

La siguiente tabla consolida cómo cada requisito del apartado de seguridad, cumplimiento y LOPD se materializa en la arquitectura propuesta, cumpliendo el mandato del ejercicio de argumentar y explicar la presencia de estos elementos en la solución.

Requisito	Control	Componente C4	Justificación
Seguridad perimetral	WAF + API Gateway + rate limit	API Gateway + WAF	Filtra ataques antes de la plataforma
Identidad robusta	MFA + OAuth2 + mTLS	IAM / IdP	Autenticación fuerte por tipo de actor
Autorización granular	ABAC con claims JWT	Policy Enforcement	Cada actor solo accede a lo permitido
LOPD consentimiento	Validación consent_id	Policy Enforcement	Tratamiento lícito de datos de terceros
LOPD minimización	Tokenización, sin PII en eventos	Event Publisher	Reduce riesgo en mensajería

Requisito	Control	Componente C4	Justificación
LOPD derechos del titular	APIs acceso, rectificación, borrado, portabilidad	customer-integration-svc	Cumple derechos ARCO+
Cumplimiento Open Finance	Scopes, mTLS, auditoría de accesos	API Gateway, Audit Trail	Adherencia regulatoria verificable
AML / KYC	Validación pre-operación	Adaptador AML	Prevención lavado de activos
Auditoría regulador	Logs inmutables 7 años	OpenSearch, Audit Trail	Evidencia para inspecciones
Segregación de funciones	Roles RBAC mutuamente excluyentes	IAM, Policy Enforcement	Nadie concentra permisos incompatibles
Seguridad en CI/CD	SAST, DAST, escaneo imágenes	Pipeline CI/CD	Código vulnerable no llega a producción

ADR-016: Privacy by design y controles LOPD integrados en arquitectura

Contexto: La LOPD exige protección de datos desde el diseño, no como parche posterior. Los eventos en Kafka, los logs y las APIs de terceros son vectores de exposición de PII.

Decisión: Implementar privacy by design: minimización de PII en eventos (customer_token), ABAC para acceso a datos propios, consent_id obligatorio para terceros, APIs de derechos del titular y políticas de retención automática.

Justificación: Cumple principios LOPD de licitud, minimización, limitación de finalidad y derechos ARCO+. Los controles estan en componentes identificables (Policy Enforcement, Audit Trail) y son auditables.

Alternativas descartadas: Anonimizar todo: descartado porque el negocio bancario requiere identificación del titular. Cifrar PII en eventos sin tokenizar: descartado por riesgo de filtración en logs downstream.

Referencia C4: Componentes: Policy Enforcement, Audit Trail; Contenedores: IAM, OpenSearch

7. Estrategia de integración multicore

Durante la modernización, el core tradicional y el core digital coexisten. Los canales no deben saber en qué core reside cada producto. El componente Core Router Multicore (C4 Contenedores y Componentes) implementa reglas de enrutamiento configurables en base de datos o ConfigMap.

7.1 Matriz de sistemas de registro (systems of record)

Durante la convivencia multicore, cada entidad de negocio tiene un **system of record (SoR)** que es la fuente autoritativa de verdad. La matriz define el SoR actual, el SoR objetivo tras la migración y la fase en la que cambia, evitando ambigüedad sobre donde persiste cada dato.

Entidad de negocio	Dominio BIAN	SoR actual	SoR objetivo	Fase migración	Notas
Cuenta corriente historica	Current Account	Core legacy (on-prem)	Core digital	Fase 2-4	Router: PRODUCT_LEGACY → PRODUCT_MIGRATION D
Cuenta ahorro digital (nueva)	Current Account	Core digital	Core digital	Ya en digital	Router: PRODUCT_MIGRATION D desde alta
Saldo y movimientos	Current Account	Según producto (legacy o digital)	Core digital	Fase 2-3	Consulta AGGREGATION si cliente en migración
Cliente / perfil KYC	Customer Offer	Core digital + AML	Core digital	Fase 2	customer-integration-svc orquesta ambos
Transferencia / pago	Payment Execution	Core legacy + sistema pagos	Core digital + pagos	Fase 3-4	DUAL_WRITE durante migración activa
Consentimiento Open Finance	Open Banking	customer-integration-svc (BD)	customer-integration-svc	Fase 3	SoR del consentimiento no del core
Alerta / caso fraude	Fraud Evaluation	Motor de fraude	Motor de fraude	Sin migración	SoR externo; integración por eventos

Entidad de negocio	Dominio BIAN	SoR actual	SoR objetivo	Fase migración	Notas
Scoring / límites riesgo	Risk Management	Motor de riesgos	Motor de riesgos	Fase 4	gRPC sync; no depend del core
Orden de pago ISO 20022	Payment Execution	Sistema de pagos	Sistema de pagos	Sin cambio SoR	Adaptador encapsula protocolo

7.2 Reglas de enrutamiento del Core Router

Regla	Condición	Destino	Ejemplo
PRODUCT_MIGRATED	product_code en lista migrada	Core digital	Cuenta ahorro digital 2025+
PRODUCT_LEGACY	product_code no migrado	Core legacy	Cuenta corriente historica
DUAL_WRITE	cliente en fase de migración activa	Ambos cores	Transferencia actualiza ambos
AGGREGATE	consulta saldo total	Ambos + merge	Saldo = legacy + digital

ADR-012: Strangler Fig con Core Router

Contexto: Migración big-bang del core es riesgo inaceptable para un banco en producción.

Decisión: Migrar por producto/cliente con Core Router que enruta progresivamente al core digital.

Justificación: Reduce riesgo: rollback = cambiar regla de enrutamiento. Parallel run con reconciliación batch detecta divergencias.

Alternativas descartadas: Big-bang: descartado. Replicación pasiva sin router: canales seguirían acoplados a legacy.

Referencia C4: Contenedores: Core Router Multicore

8. Estrategia de APIs internas/externas y mensajería

Este apartado responde al requisito 7: diseñar una estrategia de APIs internas y externas bajo estándares de industria respecto a la mensajería. La propuesta distingue tres canales de integración: (1) APIs externas expuestas a terceros regulados (Open Finance), (2) APIs internas entre componentes de la plataforma (BFF, microservicios, adaptadores) y (3) mensajería asíncrona basada en eventos (Kafka/MSK) para desacoplamiento y alto volumen.

El criterio rector es contract-first: todo contrato (REST, gRPC o evento) se define antes de implementar, se versiona semánticamente y se valida en CI/CD. La decisión entre API síncrona y mensajería asíncrona depende de la latencia requerida, el acoplamiento aceptable y la necesidad de replay/auditoría.

8.1 APIs externas (Open Finance y terceros regulados)

Las APIs externas se exponen exclusivamente a través del API Gateway + WAF. Siguen el estándar REST con especificación OpenAPI 3.1 publicada en el catálogo de desarrolladores. Los errores utilizan RFC 7807 (application/problem+json) para interoperabilidad con terceros.

La seguridad combina OAuth2 (Client Credentials / Authorization Code), mTLS obligatorio y scopes granulares vinculados al consentimiento del titular (ver sección 5.3). El versionado semántico en la URL (/v1/, /v2/) garantiza compatibilidad: una versión anterior permanece activa mínimo 12 meses tras publicar la siguiente.

Estándar / práctica	Aplicación	Componente C4
OpenAPI 3.1	Contrato publicado y validado en CI (Dredd/Spectral)	API Gateway
RFC 7807 (problem+json)	Errores estructurados para terceros	API Controller
OAuth2 + mTLS	Autenticación y autorización de fintechs	API Gateway, IAM
Versionado /v1/, /v2/	Deprecación controlada sin romper consumidores	API Gateway
Paginación cursor-based	Listados de cuentas/movimientos escalables	API Controller
Rate limiting por client_id	Protección contra abuso	API Gateway + WAF

8.1.1 Catálogo de APIs de ejemplo

Las APIs siguen contrato OpenAPI 3.1 publicado en Backstage. La tabla siguiente muestra ejemplos representativos de APIs externas (Open Finance) e internas (plataforma), con método, path, autorización y

dominio BIAN asociado.

Tipo	Método	Path	Scope / AuthZ	Dominio BIAN	Descripción
Externa	GET	/v1/accounts	accounts:read + consent_id	Open Banking	Listado de cuentas consentidas del titular
Externa	GET	/v1/accounts/{accountId}/balances	accounts:read	Current Account	Consulta de saldo de una cuenta
Externa	GET	/v1/accounts/{accountId}/transactions	accounts:read	Current Account	Movimientos paginados (cursor-based)
Externa	POST	/v1/payments	payments:write + consent_id	Payment Execution	Iniciar transferencia o pago
Externa	GET	/v1/customers/{customerId}	customers:read	Customer Offer	Datos del titular (sin PII excesiva)
Externa	POST	/v1/consents	consents:write	Open Banking	Otorgar consentimiento a tercero
Externa	DELETE	/v1/consents/{consentId}	consents:write	Open Banking	Revocar consentimiento vigente
Interna	GET	/internal/v1/accounts/{accountId}	IRSA (svc-to-svc)	Current Account	BFF → account-integration-svc
Interna	POST	/internal/v1/payments	IRSA + Idempotency-Key	Payment Execution	Orquestador → payment-integration-svc
Interna	PUT	/internal/v1/customers/{customerId}	IRSA	Customer Offer	Rectificación de datos (LOPD)
Interna	gRPC	RiskService.EvaluateTransfer	mTLS interno	Risk Management	Evaluación de riesgo p99 < 100 ms
Interna	POST	/internal/v1/fraud/signals	IRSA	Fraud	Envío de señal antifraude

Tipo	Método	Path	Scope / AuthZ	Dominio BIAN	Descripción
				Evaluation	síncrona

8.2 APIs internas (plataforma de integración)

Las APIs internas conectan BFF, microservicios de integración, orquestador y adaptadores. No se exponen a internet: el API Gateway usa HTTP API (interno) para canales propios y solo enruta tráfico desde BFF hacia microservicios dentro de la VPC.

Se selecciona REST para operaciones CRUD y consultas con contratos OpenAPI compartidos. Se selecciona gRPC para comunicación inter-servicio de baja latencia (ej. consulta de riesgo en tiempo real con p99 < 100ms). Los adaptadores hacia cores legacy y pagos encapsulan protocolos heterogéneos (MQ, batch, ISO 20022) tras interfaces REST/gRPC internas estándar.

- Versionado interno: header Accept-Version o prefijo /internal/v1/ (sin exposición pública).
- Propagación de contexto: headers X-Transaction-Id, X-Customer-Id, Authorization (JWT forward).
- Contract testing con Pact entre BFF y microservicios; breaking-change detection en CI.

Consumidor → Proveedor	Protocolo	Estándar contrato	Componente C4
BFF Web/Móvil → Microservicio	REST/HTTPS	OpenAPI 3.1 interno	BFF → Microservicios
Microservicio → Microservicio (sync)	gRPC	Protobuf + buf validate	Microservicios Integración
Microservicio → Orquestador	REST o evento	OpenAPI / AsyncAPI	Orquestador Saga
Orquestador → Adaptadores	REST/gRPC interno	OpenAPI por adaptador	Capa Adaptadores
Adaptador → Core legacy	MQ / batch / API propietaria	Mapeo a modelo canónico	Command/Query Adapters
Adaptador → Core digital	REST/gRPC	OpenAPI del proveedor	Command/Query Adapters
Adaptador → Pagos	REST + ISO 20022	Contrato de pagos del banco	Command Adapters

8.3 Mensajería y eventos (estándares de industria)

La mensajería asíncrona utiliza Apache Kafka (Amazon MSK) como bus de eventos. Los contratos de eventos se definen en Avro y se registran en Schema Registry (AWS Glue) con política de compatibilidad BACKWARD: un consumidor con schema v1 puede leer eventos v2 si solo se anaden campos.

La convención de nombres de topics sigue el patrón cloud-events friendly: banca.{dominio}. {evento}.v{version}. La clave de partición es account_id para garantizar orden de eventos por cuenta. Los eventos de dominio se documentan en AsyncAPI 2.6 como complemento a OpenAPI para APIs síncronas.

- Retención: 30 días en producción; 7 días en topics de alta frecuencia (señales de fraude).
- Retry: backoff exponencial 1s → 5s → 25s; circuit breaker si error rate > 50% en 1 min.

Estándar / patrón	Aplicación	Componente C4
Apache Kafka (MSK)	Bus de eventos de dominio	Event Bus
Avro + Schema Registry	Contratos versionados y validados	Schema Registry
CloudEvents (atributos)	metadata: type, source, id, time en headers Kafka	Event Publisher
Outbox Pattern	Consistencia BD + evento (ADR-008)	Event Publisher
Dead Letter Queue (DLQ)	banca.{dominio}.DLQ tras 3 reintentos	Error Handler + DLQ
Idempotencia consumidor	Idempotency-Key / offset commit	Event Consumer

8.3.1 Catálogo de eventos principales (productores y consumidores)

Los eventos de dominio siguen la convención de direccionamiento por topic y se documentan en AsyncAPI 2.6 con su contrato Avro registrado en Schema Registry antes de publicación. La tabla siguiente consolida los eventos principales de la plataforma, alineados a dominios BIAN.

Evento	Topic (direccionamiento)	Productor	Consumidor(es)
AccountCreated	banca.cuentas.AccountCreated.v1	account-integration-svc	customer-integration-svc, adaptador legacy
BalanceUpdated	banca.cuentas.BalanceUpdated	account-integration-svc	fraud-integration-svc,

Evento	Topic (direccionamiento)	Productor	Consumidor(es)
	.v1		reporting batch
PaymentExecuted	banca.pagos.PaymentExecuted.v1	payment-integration-svc	fraud-integration-svc, notificaciones, risk-integration-svc
PaymentFailed	banca.pagos.PaymentFailed.v1	payment-integration-svc (Saga)	orquestador Saga, audit Trail, alerting SRE
TransactionSignal	banca.fraud.TransactionSignal.v1	payment-integration-svc	fraud-integration-svc
FraudAlertRaised	banca.fraud.FraudAlertRaised.v1	fraud-integration-svc	motor de fraude, audit Trail, caso gestión
CustomerKYCUpdated	banca.clientes.CustomerKYCUpdated.v1	customer-integration-svc	adaptador AML, risk-integration-svc
ConsentChanged	banca.openfinance.ConsentChanged.v1	customer-integration-svc	Policy Enforcement, audit Trail, API Gateway cache
NotificationRequested	banca.notificaciones.NotificationRequested.v1	payment-integration-svc, account-integration-svc	servicio notificaciones (SMS/push/email)

8.3.2 Payloads de referencia (contratos Avro)

Cada evento publica un payload Avro versionado en Schema Registry. Los atributos CloudEvents (type, source, id, time, datacontenttype) viajan en headers Kafka; el cuerpo sigue el schema Avro. No se incluye PII en texto plano: se usa customer_token tokenizado (ver ADR-016).

Evento	Campos Avro obligatorios (payload de referencia)	Headers Kafka / CloudEvents
AccountCreated	event_id (UUID), transaction_id, account_id,	ce_type, ce_source, ce_id,

Evento	Campos Avro obligatorios (payload de referencia)	Headers Kafka / CloudEvents
	customer_token, product_code, currency, opened_at (timestamp), core_source (legacy digital)	ce_time, traceparent, X-Transaction-Id
PaymentExecuted	event_id, transaction_id, account_id, customer_token, amount (decimal), currency, payment_ref, status (COMPLETED), executed_at, idempotency_key	ce_type, ce_source, ce_id, ce_time, traceparent, X-Transaction-Id
TransactionSignal	event_id, transaction_id, account_id, customer_token, amount, channel (web mobile openfinance), merchant_id, geo_country, risk_score_hint	ce_type, ce_source, ce_correlationid, traceparent
ConsentChanged	event_id, transaction_id, customer_token, consent_id, action (GRANT REVOKE), scopes[], effective_at, third_party_client_id	ce_type, ce_source, ce_id, ce_time, X-Transaction-Id
Patrón resiliencia	Comportamiento	Componente C4
Reintento consumidor	3 intentos: backoff 1s → 5s → 25s	Error Handler
Dead Letter Queue	Mensaje a banca.{dominio}.DLQ tras agotar reintentos	Error Handler + DLQ
Ordenamiento	Misma partition key (account_id) garantiza orden por cuenta	Event Bus (Kafka)
Idempotencia	Consumidor deduplica por event_id + idempotency_key en Redis	Event Consumer, Idempotency Manager
Publicación fiable	Outbox Pattern: BD y evento en misma transacción	Event Publisher (ADR-008)

8.4 Matriz de decisión: API síncrona vs mensajería asíncrona

La estrategia define criterios claros para elegir el mecanismo de integración. No todo debe ser evento ni todo debe ser REST: la decisión depende del caso de uso.

Caso de uso	Mecanismo	Protocolo / estándar	Justificación
Consulta saldo en canal digital	API síncrona	REST + OpenAPI	Respuesta inmediata < 500ms
Evaluación riesgo en transferencia	API síncrona	gRPC	Latencia p99 < 100ms
Senal antifraude post-transacción	Mensajería asíncrona	Kafka + Avro	No bloquear respuesta al cliente
Notificación SMS/push	Mensajería asíncrona	Kafka + Avro	Fire-and-forget, alto volumen
Sincronización multicore (AccountCreated)	Mensajería asíncrona	Kafka + Avro	Desacoplar cores legacy y digital
Open Finance: consulta cuentas	API síncrona externa	REST + OpenAPI 3.1	Tercero espera respuesta HTTP
Validación AML (> 2s)	Request-Reply Kafka	Kafka + correlación	Timeout controlado sin bloquear thread
Orquestación pago con compensación	Saga (sync + eventos)	Step Functions + Kafka	Coordina sync y async

8.5 Tabla resumen: estrategia API/mensajería → solución

Consolidación de como la estrategia de APIs y mensajería se materializa en la arquitectura, cumpliendo el mandato del ejercicio de diseñar bajo estándares de industria.

Tipo integración	Estándar industria	Tecnología	Componente C4
API externa Open Finance	OpenAPI 3.1, OAuth2, mTLS, RFC 7807	API Gateway REST	API Gateway + WAF
API interna CRUD	OpenAPI 3.1, contract-first	REST/HTTPS	API Controller, BFF
API interna baja latencia	Protobuf, gRPC health checks	gRPC	Microservicios Integración

Tipo integración	Estándar industria	Tecnología	Componente C4
Evento de dominio	Avro, Schema Registry, CloudEvents	Kafka/MSK	Event Publisher/Consumer
Contrato adaptador legacy	Modelo canónico BIAN	MQ/batch/API	Command/Query Adapters
Pagos ISO 20022	ISO 20022 + mapeo interno	REST adaptador	Command Adapters → Pagos
Resiliencia mensajería	DLQ, retry, idempotencia	Kafka + Redis	Error Handler, Idempotency Manager

ADR-019: Criterios de selección REST vs gRPC vs mensajería Kafka

Contexto: La plataforma mezcla canales síncronos (consultas, pagos) y procesos asíncronos (fraude, notificaciones). Elegir mal el mecanismo genera latencia excesiva o acoplamiento frágil.

Decisión: REST para APIs externas y CRUD interno; gRPC para inter-servicio sync de baja latencia; Kafka para eventos de dominio, notificaciones y sincronización multicore; Request-Reply Kafka solo cuando la operación supera 2s pero no requiere respuesta HTTP al cliente.

Justificación: Alinea con estándares de industria (OpenAPI, Avro, CloudEvents) y optimiza cada caso: REST maximiza interoperabilidad externa; gRPC minimiza latencia interna; Kafka maximiza desacoplamiento y replay.

Alternativas descartadas: Solo REST para todo: descartado por latencia en riesgo/fraude. Solo eventos: descartado para consultas que requieren respuesta inmediata al usuario.

Referencia C4: API Gateway, Microservicios, Event Bus, Schema Registry

9. Gobierno de APIs y microservicios

Este apartado responde al requisito 8: proponer un modelo de gobierno de APIs y microservicios. El gobierno asegura que cada API y cada microservicio tenga ownership claro, pase por controles de calidad y seguridad antes de producción, y evolucione de forma controlada (versionado, deprecación, retiro).

El modelo se organiza en tres dimensiones: (1) gobierno organizativo (roles y RACI), (2) gobierno de APIs (ciclo de vida, catálogo, KPIs) y (3) gobierno de microservicios y eventos (estándares EKS, namespaces BIAN, Schema Registry). La plataforma API Gateway y el catálogo Backstage son los puntos de control central.

9.1 Organización, roles y matriz RACI

Cada API y microservicio tiene un owner de negocio (API Product Owner) responsable del roadmap y un owner técnico (API Champion / Tech Lead) responsable del diseño y la calidad del contrato. La matriz RACI define responsabilidades para evitar ambigüedad en decisiones de publicación, cambios breaking y retiro de servicios.

Rol	Responsabilidad			Ejemplo en la arquitectura
API Product Owner (negocio)	Define requisitos, prioriza roadmap, aprueba deprecación			Dueño de API Open Finance cuentas
API Champion (arquitectura)	Valida diseño, contrato OpenAPI, alineación BIAN			Revisa account-integration-svc
API Developer (equipo)	Implementa, testea, despliega			Desarrolla payment-integration-svc
Plataforma / SRE	Opera EKS, MSK, observabilidad, SLOs			Gestiona cluster y alertas
Seguridad	Threat model, políticas IAM, escaneo			Aprueba exposición API externa
Cumplimiento	Valida adherencia regulatoria Open Finance			Aprueba scopes y consentimiento
Actividad	R (Responsable)	A (Aprueba)	C (Consultado)	I (Informado)
Diseño de nueva API	API Champion	ARB	Seguridad, Cumplimiento	Product Owner
Publicación en catálogo	API Developer	API Champion	Plataforma	Consumidores

Actividad	R (Responsable)	A (Aprueba)	C (Consultado)	I (Informado)
Cambio breaking en contrato	API Developer	ARB + Product Owner	Consumidores	SRE
Deprecación / retiro API	Product Owner	ARB	Consumidores, Cumplimiento	SRE
Nuevo microservicio EKS	Tech Lead dominio	ARB	Plataforma, Seguridad	Product Owner
Nuevo topic Kafka	API Developer	API Champion	Plataforma	Consumidores evento

9.2 Ciclo de vida de APIs

Toda API (externa o interna) sigue un ciclo de vida formal con gates de aprobación. No se permite publicar una API en producción sin contrato OpenAPI validado, pruebas de contrato (Pact/Dredd) y revisión de seguridad.

Fase	Actividades	Gate / criterio de salida	Herramienta
1. Diseño	Definir OpenAPI, scopes, SLAs, modelo de errores RFC 7807	ARB aprueba diseño	Swagger Editor, ARB
2. Implementación	Código, tests unitarios, contract tests	CI verde, coverage > 80%	GitHub Actions, Pact
3. Revisión seguridad	Threat model, OAuth2/mTLS, rate limits	Seguridad aprueba	OWASP ZAP, Spectral
4. Publicación	Registrar en catálogo, desplegar en API GW	OpenAPI publicado, versión /v1/	Backstage, API GW
5. Operación	Monitorear KPIs, atender incidentes	SLO cumplido 30 días	Grafana, PagerDuty
6. Deprecación	Anunciar sunset 12 meses, mantener /v1/	0 consumidores en	Developer Portal

Fase	Actividades	Gate / criterio de salida	Herramienta
	paralelo	versión vieja	
7. Retiro	Eliminar rutas, archivar contrato	ARB + Product Owner aprueban	GitOps, API GW

9.3 Gobierno de microservicios

Los microservicios de integración se organizan por dominio BIAN en namespaces de Kubernetes dedicados. Cada namespace tiene resource quotas, network policies y un owner técnico. Los estándares de despliegue se validan con policy-as-code antes de llegar a producción.

Estándar	Regla	Herramienta	Componente C4
Naming	svc: {dominio}-integration-svc	Convención Git	Microservicios Integración
Namespace	ns-{dominio} (accounts, payments, fraud)	Kubernetes	EKS
Labels obligatorios	app, version, owner, domain-bian	OPA/ Gatekeeper	EKS
Replicas mínimas	≥ 3 en producción, PDB minAvailable 2	HPA, PDB	EKS
Imágenes	Escaneo Trivy, firma Cosign antes de ECR	CI/CD pipeline	ECR
Secrets	Nunca en código; solo Secrets Manager	OPA + CI lint	Vault / Secrets Manager
Observabilidad	OTel obligatorio, métricas RED (Rate, Errors, Duration)	ADOT sidecar	CloudWatch, X-Ray
GitOps	Todo cambio via PR + ArgoCD sync	ArgoCD	EKS primario y DR

9.4 Gobierno de eventos y contratos (Schema Registry)

Los eventos de dominio siguen el mismo rigor de gobierno que las APIs: contrato Avro registrado, versionado con compatibilidad BACKWARD y ownership por dominio BIAN. Un cambio breaking en un evento requiere aprobación del API Champion y notificación a todos los consumidores del topic.

- Registro obligatorio en Schema Registry antes de publicar en producción.

- Convención topic: banca.{dominio}.{evento}.v{version} — ownership = equipo del dominio.
- AsyncAPI 2.6 como documentación complementaria a OpenAPI para cada evento.
- Consumidores deben declarar dependencia en catálogo Backstage (producer → consumer graph).

9.5 KPIs, tableros y mejora continua

KPI	Objetivo	Fuente	Responsable
p95 latencia API externa	< 500 ms	CloudWatch / Grafana	SRE
Error rate 5xx	< 0.1%	API Gateway metrics	API Developer + SRE
Disponibilidad mensual API	≥ 99.95%	SLO burn-rate alerts	SRE
Adopción (calls/consumer)	Crecimiento o estabilidad	API GW logs	Product Owner
MTTR incidentes API	< 30 min	PagerDuty	SRE
Contratos sin breaking change no autorizado	0 en producción	CI Spectral/Pact	API Champion
Microservicios sin labels obligatorios	0 en cluster	OPA audit	Plataforma

9.6 Tabla resumen: gobierno → implementación en la arquitectura

Dimensión gobierno	Mecanismo	Componente / herramienta C4
Catálogo y descubrimiento	Backstage / API GW Developer Portal	API Gateway
Control de publicación	ARB + gates CI/CD	Pipeline, ARB
Versionado API	/v1/, /v2/ en API Gateway	API Gateway + WAF
Contract testing	Pact (interno), Dredd (externo)	CI/CD
Seguridad API	Spectral lint + OWASP ZAP	API Gateway, WAF

Dimensión gobierno	Mecanismo	Componente / herramienta C4
Gobierno microservicios	OPA/Gatekeeper, namespaces BIAN	EKS
Gobierno eventos	Schema Registry, AsyncAPI	Event Bus, Schema Registry
Observabilidad y SLOs	Grafana dashboards por API/svc	CloudWatch, OpenSearch

ADR-020: Modelo de gobierno centralizado con federación por dominio BIAN

Contexto: Sin gobierno, la proliferación de APIs y microservicios genera deuda técnica, contratos inconsistentes y riesgo regulatorio en Open Finance.

Decisión: Establecer gobierno centralizado (ARB, catálogo Backstage, políticas OPA) con federación por dominio BIAN: cada equipo es owner de sus APIs y eventos, pero debe cumplir estándares corporativos y pasar gates antes de producción.

Justificación: Combina autonomía de equipos (federación) con control corporativo (centralizado). El catálogo único evita APIs duplicadas; OPA enforce estándares sin revisión manual en cada deploy.

Alternativas descartadas: Gobierno totalmente centralizado (un equipo aprueba todo): cuello de botella. Sin gobierno (cada equipo libre): caos y deuda técnica.

Referencia C4: API Gateway, Microservicios Integración, Schema Registry, EKS

10. Plan de migración gradual

Este apartado responde al requisito 9: proponer un plan de migración gradual que minimice el riesgo operativo. La modernización del core bancario no puede ejecutarse como un big-bang: una interrupción de servicio afecta a millones de clientes, expone al banco a sanciones regulatorias y compromete la confianza en canales digitales.

El plan se apoya en el patrón **Strangler Fig** (ADR-012) y en el **Core Router Multicore**: los canales mantienen contratos estables mientras el tráfico se desvía progresivamente al core digital por producto, cliente o dominio BIAN. Cada fase tiene entregables medibles, riesgos identificados, controles transversales y criterios go/no-go antes de avanzar.

10.1 Principios de minimización de riesgo operativo

- **Convivencia controlada:** legacy y digital operan en paralelo; el router decide el destino sin cambiar contratos de canal.
- **Migración incremental:** por dominio BIAN y por producto, no por corte global de fecha.
- **Parallel run:** durante dual-write, ambos cores procesan la misma operación; reconciliación batch detecta divergencias.
- **Rollback reversible:** revertir una migración = actualizar regla del Core Router (segundos), no redeploy de canales.
- **Observabilidad antes de tráfico:** dashboards, alertas y SLOs activos antes de mover clientes reales.
- **Gates formales:** comité de migración (ARB + SRE + negocio) aprueba avance de fase con métricas objetivas.

10.2 Fases del plan de migración

Fase	Duración	Entregables clave	Riesgo mitigado	Criterio go/no-go
1 Fundación	0-3 meses	EKS, API GW, Cognito, MSK, CI/CD GitOps, observabilidad (OTel, Grafana)	Despliegue manual e inseguro; falta de trazabilidad	Pipeline despliega en < 15 min; cluster HA con 3 AZ
2 Adaptadores + piloto	3-6 meses	Adaptadores legacy/digital, Core Router, dominio cuentas (BIAN), eventos no críticos	Acoplamiento directo canal → legacy; canales conocen multicore	Consulta saldo multicore funcional; 0 regresiones en piloto
3 Migración por producto	6-12 meses	Canary releases, dual-write controlado, APIs Open	Interrupción de servicio; pérdida de transacciones	Error rate < 0.1%, p95 < 500 ms, reconciliación <

Fase	Duración	Entregables clave	Riesgo mitigado	Criterio go/no-go
		Finance, saga con compensación		0.01% divergencia
4 Consolidación	12-18 meses	Retiro integraciones punto a punto, optimización costes, decommission parcial legacy	Deuda técnica; dependencia de sistemas obsoletos	0 integraciones P2P activas; 80% tráfico en core digital
5 Estabilización	18+ meses	100% core digital, legacy solo lectura/archivo, runbooks de cierre	Datos huérfanos en legacy; incumplimiento regulatorio post-cierre	Reconciliación sin divergencias 90 días; auditoría regulatoria OK

10.3 Controles transversales (todas las fases)

Control	Descripción	Herramienta	Componente C4
Parallel run	Ejecutar misma operación en legacy y digital; comparar resultados	Jobs batch + reconciliación	Core Router, Adaptadores
Feature flags	Activar/desactivar ruta multicore por producto o % clientes sin redeploy	AWS AppConfig / LaunchDarkly	Core Router Multicore
Canary / blue-green	Desviar 1-5% tráfico al core digital antes de ampliar	API GW weighted routing	API Gateway, EKS
Dual-write controlado	Solo productos en migración activa; compensación saga si un core falla	Orquestador Saga	Saga Orchestrator
Reconciliación automática	Batch nocturno detecta divergencias saldo/movimientos	Glue / Step Functions	Event Bus, Adaptadores
Rollback probado	Runbook trimestral: revertir regla router y validar en < 5 min	GitOps + ConfigMap	Core Router Multicore
Freeze de cambios	Ventana de 48 h sin deploys no críticos antes de mover clientes	Calendario ARB	CI/CD, ArgoCD

10.4 Orden de migración por dominio BIAN

La secuencia prioriza dominios de menor criticidad transaccional y mayor valor de aprendizaje, dejando

pagos y fraude para cuando la plataforma y los controles esten maduros.

Orden	Dominio BIAN	Microservicio	Fase objetivo	Razón del orden
1	Customer Offer	customer-integration-svc	Fase 2	Consultas de bajo riesgo; valida adaptadores
2	Current Account	account-integration-svc	Fase 2-3	Piloto multicore; saldo agregado legacy+digital
3	Open Banking	openfinance-integration-svc	Fase 3	Regulatorio; requiere APIs estables y consentimiento
4	Payment Execution	payment-integration-svc	Fase 3-4	Alto impacto; exige saga, idempotencia y reconciliación
5	Fraud Evaluation	fraud-integration-svc	Fase 3-4	Asíncrono; depende de eventos de pagos
6	Risk Management	risk-integration-svc	Fase 4	Scoring en tiempo real; requiere gRPC maduro

10.5 Comité de migración, go/no-go y procedimiento de rollback

Un **Comité de Migración** (API Champion, SRE, Product Owner, Cumplimiento) se reúne al cierre de cada fase. No se avanza sin cumplir todos los criterios go/no-go de la tabla 10.2 y sin rollback probado en el entorno de preproducción.

- **Go:** métricas en verde 7 días consecutivos, 0 incidentes P1 en dominio migrado, reconciliación OK.
- **No-go / pause:** error rate > 0.1%, divergencia reconciliación > 0.01%, o incidente P1 sin RCA cerrado.
- **Rollback (procedimiento):** (1) desactivar feature flag del producto, (2) revertir regla Core Router a PRODUCT_LEGACY, (3) drenar cola Kafka del dominio, (4) notificar canales via status page, (5) post-mortem en 48 h.

10.6 Tabla resumen: requisito 9 → implementación en la arquitectura

Necesidad del requisito 9	Mecanismo	Componente / herramienta C4
Migración gradual sin big-bang	Strangler Fig por producto/cliente	Core Router Multicore
Minimizar riesgo operativo	Parallel run + reconciliación + rollback	Adaptadores, Saga, Event Bus
Sin interrupción de canales	Contratos API estables en API GW	API Gateway, BFF
Trazabilidad de cambios	GitOps, feature flags, auditoría	ArgoCD, AppConfig, Audit Trail
Cumplimiento en migración	Consentimiento y scopes sin cambio para terceros	IAM, Policy Enforcement
Gobernanza del avance	Comité migración + gates por fase	ARB, KPIs Grafana

ADR-021: Plan de migración por fases con controles transversales

Contexto: La migración del core bancario implica riesgo operativo, regulatorio y reputacional. Un corte big-bang o la ausencia de controles de rollback invalida la propuesta ante un evaluador y ante la operación real del banco.

Decisión: Ejecutar migración en 5 fases (fundación → adaptadores → migración por producto → consolidación → estabilización) con controles transversales obligatorios: parallel run, feature flags, canary, dual-write con saga, reconciliación batch y rollback probado del Core Router. Secuencia por dominio BIAN de menor a mayor criticidad transaccional.

Justificación: Cada fase entrega valor incremental y reduce superficie de riesgo. Los controles transversales son reutilizables en todas las fases. El rollback por regla de enrutamiento evita redeploy de canales y limita MTTR a minutos.

Alternativas descartadas: Big-bang fin de semana: descartado por riesgo inaceptable y ventana regulatoria. Migración solo por fecha sin router: descartado porque canales seguirían acoplados a legacy.

Referencia C4: Core Router Multicore, Adaptadores, Saga Orchestrator, API Gateway, Event Bus

11. Modelo de diseño de observabilidad y trazabilidad

Este apartado responde al requisito 10: proponer un modelo de diseño para garantizar la observabilidad y la trazabilidad. En una plataforma de integración bancaria con decenas de microservicios, dos cores coexistentes y canales síncronos y asíncronos, no basta con instalar herramientas: se requiere un modelo coherente que permita (1) operar con SLOs medibles, (2) diagnosticar incidentes en minutos y (3) reconstruir cualquier operación de negocio para auditoría y regulador.

El modelo distingue dos dimensiones complementarias: **observabilidad operativa** (telemetría técnica para SRE: métricas, logs estructurados y trazas distribuidas) y **trazabilidad de negocio** (evidencia inmutable de quién hizo qué, cuándo y sobre qué recurso, para cumplimiento y no repudio). Ambas comparten un identificador de correlación (**transaction-id**) propagado en toda la cadena.

11.1 Visión del modelo: tres pilares + trazabilidad de negocio

Pilar / dimensión	Qué observa	Estándar / herramienta	Componente C4
Métricas	Salud y rendimiento (latencia, errores, saturación)	Prometheus/CloudWatch; patrón RED y USE	Observabilidad + SIEM
Logs	Eventos discretos estructurados (JSON)	CloudWatch Logs; formato ECS/JSON con campos obligatorios	Microservicios, API Gateway, Audit Trail
Trazas distribuidas	Flujo request end-to-end entre servicios y cores	OpenTelemetry (ADOT) → AWS X-Ray	EKS pods, Adaptadores, Core Router
Trazabilidad de negocio	Evidencia regulatoria: actor, acción, recurso, consentimiento	Audit Trail → OpenSearch (retención 7 años, ISM)	Audit Trail, Auditoría/Trazabilidad

11.2 Identificadores de correlación y propagación de contexto

Todo request que entra por API Gateway recibe o genera un **transaction-id** (UUID v4). Este identificador se propaga de forma obligatoria en toda la cadena síncrona y asíncrona para permitir correlación en dashboards, trazas y auditoría.

- **Regla HTTP:** ningún microservicio responde sin haber registrado transaction-id en log estructurado.
- **Regla Kafka:** todo mensaje de dominio incluye transaction-id y trace-id en headers del record.
- **Regla Saga:** Step Functions persiste transaction-id en cada transición de estado para reconstrucción.
- **Regla Core Router:** cada decisión de enrutamiento (legacy/digital/dual-write) se registra en Audit

Trail con transaction-id.

Identificador	Origen	Propagación	Uso
transaction-id	API Gateway (genera si ausente)	Header HTTP X-Transaction-Id; baggage OTel; header Kafka; campo saga Step Functions	Correlación logs + trazas + auditoría end-to-end
trace-id / span-id	SDK OpenTelemetry en cada pod	W3C Trace Context (traceparent) en HTTP; inyectado en mensajes Kafka	Trazas distribuidas en X-Ray
customer-id / customer_token	JWT Cognito o token interno	Header X-Customer-Id (interno); customer_token en eventos (sin PII)	ABAC, partición Kafka, búsqueda en auditoría
correlation-id (eventos)	Event Publisher (CloudEvents)	Atributo ce_correlationid en payload Avro	Replay y depuración de flujos asíncronos

11.3 Arquitectura de recolección y almacenamiento

La recolección sigue el patrón **instrumentación unificada con OpenTelemetry**: cada pod EKS ejecuta el AWS Distro for OpenTelemetry (ADOT) como sidecar o daemonset que exporta métricas, logs y trazas a los backends gestionados de AWS, minimizando agentes propietarios heterogéneos.

Capa	Qué instrumenta	Exporta a	Retención
Canales / BFF	Latencia por pantalla, errores 4xx/5xx	CloudWatch + X-Ray	30 días logs; 90 días trazas
API Gateway	Throttling, latencia p50/p95/p99, 5xx por ruta	CloudWatch metrics + access logs	90 días access logs
Microservicios EKS	RED por servicio; logs JSON estructurados	ADOT → CloudWatch + X-Ray	30 días logs operativos
Core Router / Saga	Decisiones enrutamiento; pasos compensación	Audit Trail + X-Ray	7 años auditoría; 90 días trazas
Kafka / MSK	Lag consumidor, throughput, errores DLQ	CloudWatch MSK metrics + logs broker	30 días broker logs

Capa	Qué instrumenta	Exporta a	Retención
Adaptadores legacy	Latencia hacia on-prem; timeouts; errores	ADOT + Audit Trail	7 años auditoría operaciones
OpenSearch	Índice auditoría + búsqueda full-text compliance	OpenSearch cluster (3 nodos)	7 años ISM policy

11.4 SLOs, SLIs y alertas operativas

Los SLOs se definen por API expuesta y por microservicio crítico. Las alertas usan burn-rate sobre ventanas de 1 h y 6 h para detectar degradación antes de violar el objetivo mensual.

Servicio / API	SLI	SLO objetivo	Alerta (PagerDuty)
API Gateway (externa)	Disponibilidad (excl. 4xx cliente)	99.95% mensual	Burn-rate > 14x en 1 h
payment-integration-svc	p95 latencia end-to-end	< 500 ms	p95 > 500 ms durante 5 min
account-integration-svc	Tasa error 5xx	< 0.1%	Error rate > 0.1% durante 5 min
Core Router Multicore	Decisiones enrutamiento correctas	100% (reconciliación)	Divergencia > 0.01% en batch
MSK / consumidores	Consumer lag	< 1000 mensajes	Lag > 5000 durante 10 min
Saga pagos	Compensaciones exitosas	> 99.9%	Compensación fallida → P1 inmediato

11.5 Trazabilidad de negocio y cumplimiento (Audit Trail)

La trazabilidad de negocio es distinta de la observabilidad operativa: su proposito es generar evidencia verificable para el regulador, investigaciones de fraude y cumplimiento Open Finance. El componente **Audit Trail** (C4 Componentes) centraliza eventos de negocio inmutables.

- **Inmutabilidad:** índices OpenSearch con ISM; sin permisos de delete para roles operativos.
- **SIEM:** CloudWatch Logs subscription filter → alertas de patrones de seguridad correlacionados con transaction-id.
- **Regulador:** APIs de cumplimiento (rol compliance_reader) exportan evidencias sin acceso a producción.

Evento auditado	Campos obligatorios	Quién lo genera	Retención
Acceso API externa (tercero)	actor, consent_id, scope, resource, transaction-id, timestamp, IP	API Gateway → Audit Trail	7 años
Operación de pago	transaction-id, monto, cuenta_origen, cuenta_destino, resultado, saga_step	payment-integration-svc, Saga Orchestrator	7 años
Decision Core Router	transaction-id, regla aplicada, destino (legacy/digital/both), product_code	Core Router Multicore	7 años
Cambio de consentimiento	customer_id, consent_id, acción (grant/revoke), timestamp	Policy Enforcement	7 años
Acceso privilegiado empleado	user_id, rol, recurso, acción, IP, MFA_verified	IAM Identity Center → Audit Trail	7 años

11.6 Dashboards, runbooks y flujo de diagnóstico

Grafana (sobre CloudWatch y OpenSearch) provee tableros por dominio BIAN y por capa. El flujo de diagnóstico ante incidente P1 sigue: (1) alerta PagerDuty, (2) localizar transaction-id afectado, (3) reconstruir traza en X-Ray, (4) cruzar logs estructurados y eventos Audit Trail, (5) identificar componente root cause, (6) ejecutar runbook (rollback router, drain DLQ, etc.).

Dashboard	Audiencia	Métricas clave	Fuente
Plataforma integración (global)	SRE / Plataforma	Disponibilidad, p95, error rate, lag Kafka	CloudWatch, Grafana
APIs Open Finance	Product Owner + Cumplimiento	Calls por tercero, 4xx/5xx, consentimientos	API GW + Audit Trail
Multicore / migración	ARB + SRE	% tráfico digital, divergencias reconciliación	Core Router + batch jobs
Seguridad / SIEM	SOC	Intentos acceso denegado, anomalías, MFA failures	CloudWatch Logs + OpenSearch
Cumplimiento / auditoría	Oficial cumplimiento	Consultas por transaction-id,	OpenSearch

Dashboard	Audiencia	Métricas clave	Fuente
		exportes regulador	

11.7 Tabla resumen: requisito 10 → implementación en la arquitectura

Necesidad del requisito 10	Mecanismo	Componente / herramienta C4
Observabilidad operativa (SRE)	Tres pilares: métricas RED, logs JSON, trazas OTel	Observabilidad + SIEM, ADOT, CloudWatch, X-Ray
Trazabilidad end-to-end	transaction-id propagado HTTP + Kafka + saga	API Gateway, microservicios, Event Bus
Trazabilidad de negocio / regulador	Audit Trail inmutable 7 años	Audit Trail, Auditoría/Trazabilidad (OpenSearch)
Detección proactiva	SLO burn-rate + alertas PagerDuty	Grafana, CloudWatch Alarms
Diagnóstico de incidentes	X-Ray trace + logs correlacionados + runbooks	X-Ray, OpenSearch, Grafana
Cumplimiento Open Finance	Log acceso tercero con consent_id y scope	API Gateway, Policy Enforcement, Audit Trail

ADR-022: OpenTelemetry unificado con Audit Trail separado para trazabilidad de negocio

Contexto: Sin un modelo unificado, cada equipo instrumenta de forma distinta y la correlación end-to-end falla. Mezclar telemetría operativa con evidencia regulatoria en el mismo almacén complica retención (30 días vs 7 años) y cumplimiento LOPD.

Decisión: Adoptar OpenTelemetry (ADOT) como estándar único de instrumentación para métricas, logs y trazas operativas, exportando a CloudWatch y X-Ray. Mantener Audit Trail como canal separado hacia OpenSearch para trazabilidad de negocio inmutable (7 años). Correlación via transaction-id presente en ambos canales. Patrón RED obligatorio por microservicio; W3C Trace Context en HTTP y Kafka.

Justificación: OTel es estándar de industria, evita vendor lock-in en instrumentación y permite correlación distribuida. Separar auditoría de telemetría operativa respeta políticas de retención y minimización de PII en logs operativos (30 días) vs evidencia regulatoria (7 años).

Alternativas descartadas: Solo CloudWatch sin OTel: descartado por falta de trazas distribuidas estándar. Un solo almacén para todo: descartado por conflicto de retención y exposición PII en logs operativos.

Referencia C4: Observabilidad + SIEM, Audit Trail, Auditoría/Trazabilidad, EKS (ADOT), API Gateway

12. Registro consolidado de decisiones arquitectónicas (ADRs)

Resumen de las 17 decisiones arquitectónicas documentadas en este informe. Cada ADR sigue el formato: contexto, decisión, justificación, alternativas y trazabilidad C4.

ADR-001: Organización por dominio BIAN

Contexto: El banco tiene 6+ sistemas backend con equipos y ciclos de vida distintos.

Decisión: Crear microservicios de integración alineados a Service Domains BIAN, no a sistemas backend.

Justificación: Cuando el core legacy se retire, solo cambian los adaptadores del dominio afectado; los canales y contratos de API permanecen estables. Reduce el blast radius de cambios.

Alternativas descartadas: Integración por sistema (un ESB monolítico): descartado por acoplamiento y cuello de botella. Integración punto a punto: descartado por complejidad $O(n^2)$.

Referencia C4: Contenedores: Microservicios de Integración

ADR-002: Modelo canónico interno

Contexto: Core legacy representa una cuenta como registro COBOL; core digital como recurso REST JSON con estructura distinta.

Decisión: Definir un modelo canónico BIAN-aligned dentro de cada microservicio, con mappers hacia representaciones externas.

Justificación: La lógica de negocio opera sobre un único modelo; los adaptadores traducen al entrar/salir. Facilita agregación multicore (sumar saldos de ambos cores en una sola respuesta).

Alternativas descartadas: Transformación en ESB centralizado: descartado por convertir el bus en cuello de botella y punto único de fallo.

Referencia C4: Componentes: Canonical Mapper

ADR-003: API Gateway como único punto de entrada

Contexto: Canales digitales, terceros Open Finance y backoffice necesitan acceder a capacidades bancarias con políticas de seguridad homogéneas.

Decisión: Implementar Amazon API Gateway (HTTP API para internos, REST API para externos) con AWS WAF delante.

Justificación: Centraliza TLS terminación, validación JWT, rate limiting por client_id, versionado de rutas (/v1/, /v2/), logging a CloudWatch y throttling. Un único punto para aplicar parches de seguridad.

Alternativas descartadas: Kong self-hosted en EKS: mayor control pero más operación. Exposición directa de pods: inaceptable en sector regulado.

Referencia C4: Contenedores: API Gateway + WAF

ADR-004: Backend for Frontend (BFF) por canal

Contexto: La app móvil requiere payloads mínimos (menor consumo de datos); la web puede mostrar dashboards con datos agregados de múltiples dominios.

Decisión: Desplegar BFF Web y BFF Móvil como servicios independientes en EKS.

Justificación: Evita que el dominio de integración conozca detalles de UI. El BFF Móvil puede combinar 3 llamadas internas en 1 respuesta, reduciendo round-trips de 300ms a 100ms en redes móviles.

Alternativas descartadas: BFF único: descartado porque compromete optimización por canal. Sin BFF (canal llama directo a microservicios): expone complejidad y aumenta latencia.

Referencia C4: Contenedores: BFF Web, BFF Móvil

ADR-005: Arquitectura hexagonal con adaptadores

Contexto: Core legacy expone IBM MQ y archivos batch; core digital expone REST/gRPC; pagos usa ISO 20022 sobre API propietaria.

Decisión: Implementar puertos (interfaces) y adaptadores por protocolo en la Capa de Adaptadores.

Justificación: Un cambio en el protocolo del legacy (ej. migrar de MQ a Kafka) solo afecta el adaptador, no la lógica de negocio. Facilita testing con adaptadores mock.

Alternativas descartadas: ESB con transformaciones XSLT centralizadas: descartado por dificultad de versionado y testing.

Referencia C4: Contenedores: Capa de Adaptadores; Componentes: Command/Query Adapters

ADR-006: Saga Orchestration para procesos distribuidos

Contexto: Una transferencia requiere: debitar cuenta, evaluar riesgo, verificar fraude, ejecutar pago y notificar. No existe transacción ACID entre 4 sistemas heterogéneos.

Decisión: Implementar orquestación tipo Saga con AWS Step Functions para procesos críticos y máquina de estados interna para procesos simples.

Justificación: Step Functions provee estado durable, reintentos configurables, compensaciones (reverso si pago falla tras debito) y visualización del flujo. Cumple auditoría al registrar cada paso.

Alternativas descartadas: 2PC (two-phase commit): descartado porque cores legacy no lo soportan.
Coreografía pura (solo eventos): descartada para pagos por dificultad de trazabilidad y compensación.

Referencia C4: Contenedores: Orquestador Saga; Componentes: Saga Orchestrator

ADR-007: Arquitectura orientada a eventos

Contexto: El motor de fraude necesita analizar transacciones sin bloquear la respuesta al cliente. Las notificaciones (SMS/push) son inherentemente asíncronas.

Decisión: Implementar bus de eventos con Amazon MSK (Kafka gestionado), Schema Registry y topics por dominio.

Justificación: Kafka provee retención configurable (7-30 días), orden por partición (clave = account_id), throughput > 100K msg/s y replicación interregión con MirrorMaker 2. Desacopla productores de consumidores.

Alternativas descartadas: RabbitMQ: descartado para volumen bancario por limitaciones de throughput.
SQS: descartado por no garantizar orden ni replay.

Referencia C4: Contenedores: Event Bus; Componentes: Event Publisher/Consumer

ADR-008: Idempotencia y Outbox Pattern

Contexto: En pagos, un timeout de red puede provocar reintentos del cliente o del gateway, generando cargos duplicados.

Decisión: Validar Idempotency-Key en Redis (TTL 24h) antes de ejecutar. Usar Outbox Pattern: escribir evento en tabla outbox dentro de la misma transacción de negocio; un relay publica a Kafka.

Justificación: Garantiza exactamente-una-vez en efecto (at-least-once en transporte + idempotencia en consumidor). El Outbox evita inconsistencia entre BD y Kafka.

Alternativas descartadas: Confiar solo en reintentos del cliente: inaceptable en pagos. Distributed transactions (XA): no soportado entre Kafka y PostgreSQL de forma nativa.

Referencia C4: Componentes: Idempotency Manager, Event Publisher

ADR-009: Nube pública AWS como plataforma principal

Contexto: El banco necesita elasticidad, servicios gestionados y DR multiregión con time-to-market corto.

Decisión: Desplegar la plataforma de integración en AWS con topología híbrida (Direct Connect a on-prem).

Justificación: MSK y EKS son servicios maduros con SLA claros. El ecosistema de partners bancarios en AWS facilita integraciones. Direct Connect provee latencia < 5ms hacia datacenter.

Alternativas descartadas: On-prem puro: descartado por costo de operar Kafka y K8s. Multicloud activo-activo: descartado por complejidad operativa sin beneficio claro en fase inicial.

Referencia C4: Despliegue: 04-despliegue-nube

ADR-010: Cognito como IdP de clientes y terceros

Contexto: Se necesita OAuth2/OIDC nativo para web, móvil y Open Finance con MFA y escalabilidad.

Decisión: Amazon Cognito User Pools para clientes; Resource Server con scopes custom para terceros.

Justificación: Integración nativa con API Gateway (authorizer), SDK móvil (Amplify), MFA TOTP/SMS y SLA gestionado. Evita operar Keycloak en EKS.

Alternativas descartadas: Keycloak self-hosted: más flexible pero requiere operación HA. Auth0: costo elevado a escala bancaria.

Referencia C4: Contenedores: IAM/IdP

ADR-011: Autorización ABAC en Policy Enforcement

Contexto: RBAC puro (rol admin/user) es insuficiente: un cliente solo debe ver SUS cuentas; un tercero solo datos CON consentimiento.

Decisión: Implementar ABAC evaluando claims del JWT (customer_id, partner_id, scopes) contra el recurso solicitado.

Justificación: Granularidad necesaria para Open Finance y LOPD. El componente Policy Enforcement centraliza la lógica sin duplicarla en cada microservicio.

Alternativas descartadas: Autorización solo en API Gateway: insuficiente para reglas de negocio. OPA sidecar: viable en fase 2 para políticas más complejas.

Referencia C4: Componentes: Policy Enforcement

ADR-016: Privacy by design y controles LOPD integrados en arquitectura

Contexto: La LOPD exige protección de datos desde el diseño, no como parche posterior. Los eventos en Kafka, los logs y las APIs de terceros son vectores de exposición de PII.

Decisión: Implementar privacy by design: minimización de PII en eventos (customer_token), ABAC para acceso a datos propios, consent_id obligatorio para terceros, APIs de derechos del titular y políticas de retención automática.

Justificación: Cumple principios LOPD de licitud, minimización, limitación de finalidad y derechos ARCO+. Los controles estan en componentes identificables (Policy Enforcement, Audit Trail) y son auditables.

Alternativas descartadas: Anonimizar todo: descartado porque el negocio bancario requiere identificación del titular. Cifrar PII en eventos sin tokenizar: descartado por riesgo de filtración en logs downstream.

Referencia C4: Componentes: Policy Enforcement, Audit Trail; Contenedores: IAM, OpenSearch

ADR-012: Strangler Fig con Core Router

Contexto: Migración big-bang del core es riesgo inaceptable para un banco en producción.

Decisión: Migrar por producto/cliente con Core Router que enruta progresivamente al core digital.

Justificación: Reduce riesgo: rollback = cambiar regla de enrutamiento. Parallel run con reconciliación batch detecta divergencias.

Alternativas descartadas: Big-bang: descartado. Replicación pasiva sin router: canales seguirían acoplados a legacy.

Referencia C4: Contenedores: Core Router Multicore

ADR-019: Criterios de selección REST vs gRPC vs mensajería Kafka

Contexto: La plataforma mezcla canales síncronos (consultas, pagos) y procesos asíncronos (fraude, notificaciones). Elegir mal el mecanismo genera latencia excesiva o acoplamiento fragil.

Decisión: REST para APIs externas y CRUD interno; gRPC para inter-servicio sync de baja latencia; Kafka para eventos de dominio, notificaciones y sincronización multicore; Request-Reply Kafka solo cuando la operación supera 2s pero no requiere respuesta HTTP al cliente.

Justificación: Alinea con estándares de industria (OpenAPI, Avro, CloudEvents) y optimiza cada caso: REST maximiza interoperabilidad externa; gRPC minimiza latencia interna; Kafka maximiza desacoplamiento y replay.

Alternativas descartadas: Solo REST para todo: descartado por latencia en riesgo/fraude. Solo eventos: descartado para consultas que requieren respuesta inmediata al usuario.

Referencia C4: API Gateway, Microservicios, Event Bus, Schema Registry

ADR-020: Modelo de gobierno centralizado con federación por dominio BIAN

Contexto: Sin gobierno, la proliferación de APIs y microservicios genera deuda técnica, contratos inconsistentes y riesgo regulatorio en Open Finance.

Decisión: Establecer gobierno centralizado (ARB, catálogo Backstage, políticas OPA) con federación por

dominio BIAN: cada equipo es owner de sus APIs y eventos, pero debe cumplir estándares corporativos y pasar gates antes de producción.

Justificación: Combina autonomía de equipos (federación) con control corporativo (centralizado). El catálogo único evita APIs duplicadas; OPA enforce estándares sin revisión manual en cada deploy.

Alternativas descartadas: Gobierno totalmente centralizado (un equipo aprueba todo): cuello de botella. Sin gobierno (cada equipo libre): caos y deuda técnica.

Referencia C4: API Gateway, Microservicios Integración, Schema Registry, EKS

ADR-021: Plan de migración por fases con controles transversales

Contexto: La migración del core bancario implica riesgo operativo, regulatorio y reputacional. Un corte big-bang o la ausencia de controles de rollback invalida la propuesta ante un evaluador y ante la operación real del banco.

Decisión: Ejecutar migración en 5 fases (fundación → adaptadores → migración por producto → consolidación → estabilización) con controles transversales obligatorios: parallel run, feature flags, canary, dual-write con saga, reconciliación batch y rollback probado del Core Router. Secuencia por dominio BIAN de menor a mayor criticidad transaccional.

Justificación: Cada fase entrega valor incremental y reduce superficie de riesgo. Los controles transversales son reutilizables en todas las fases. El rollback por regla de enrutamiento evita redeploy de canales y limita MTTR a minutos.

Alternativas descartadas: Big-bang fin de semana: descartado por riesgo inaceptable y ventana regulatoria. Migración solo por fecha sin router: descartado porque canales seguirían acoplados a legacy.

Referencia C4: Core Router Multicore, Adaptadores, Saga Orchestrator, API Gateway, Event Bus

ADR-022: OpenTelemetry unificado con Audit Trail separado para trazabilidad de negocio

Contexto: Sin un modelo unificado, cada equipo instrumenta de forma distinta y la correlación end-to-end falla. Mezclar telemetría operativa con evidencia regulatoria en el mismo almacén complica retención (30 días vs 7 años) y cumplimiento LOPD.

Decisión: Adoptar OpenTelemetry (ADOT) como estándar único de instrumentación para métricas, logs y trazas operativas, exportando a CloudWatch y X-Ray. Mantener Audit Trail como canal separado hacia OpenSearch para trazabilidad de negocio inmutable (7 años). Correlación via transaction-id presente en ambos canales. Patrón RED obligatorio por microservicio; W3C Trace Context en HTTP y Kafka.

Justificación: OTel es estándar de industria, evita vendor lock-in en instrumentación y permite correlación distribuida. Separar auditoría de telemetría operativa respeta políticas de retención y minimización de PII en logs operativos (30 días) vs evidencia regulatoria (7 años).

Alternativas descartadas: Solo CloudWatch sin OTel: descartado por falta de trazas distribuidas estándar. Un solo almacén para todo: descartado por conflicto de retención y exposición PII en logs operativos.

Referencia C4: Observabilidad + SIEM, Audit Trail, Auditoría/Trazabilidad, EKS (ADOT), API Gateway

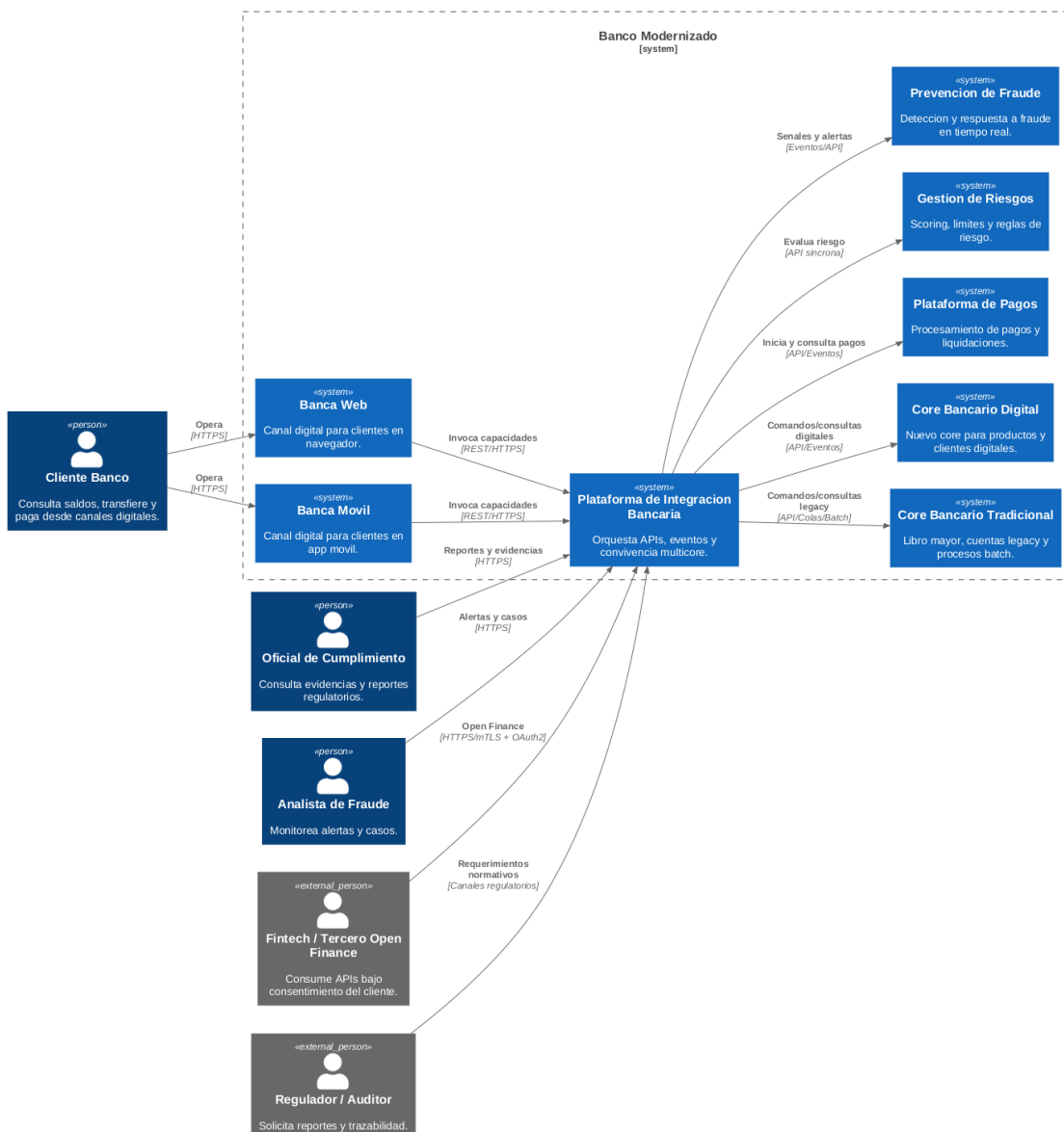
13. Diagramas C4

Los diagramas son consistentes con el documento: incluyen banca web/móvil, IAM, Core Router multicore, adaptadores legacy y despliegue en nube AWS con DR multiregión.

C4 Nivel 1 - Contexto

Actores, canales web/móvil y sistemas backend del ejercicio.

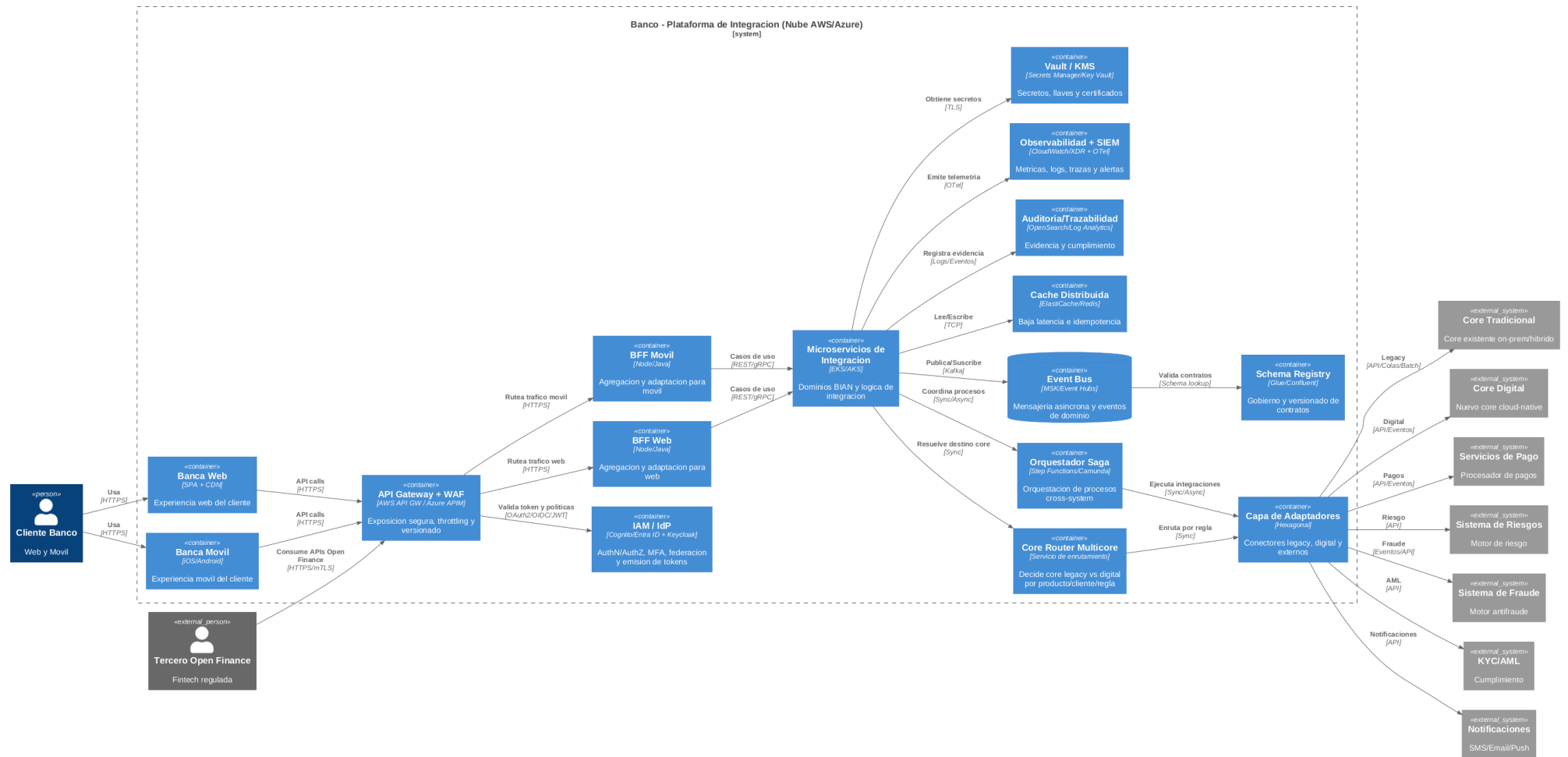
Vista ejecutiva: actores, canales web/móvil, plataforma de integración y sistemas del ejercicio.



C4 Nivel 2 - Contenedores

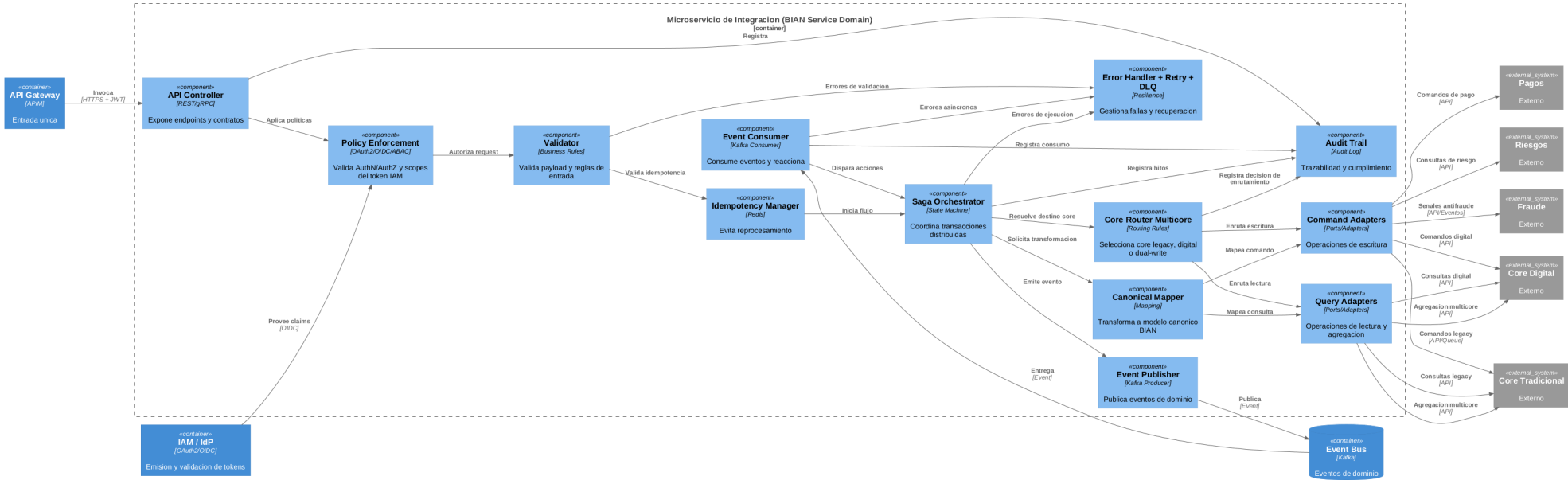
IAM, API GW, BFF, microservicios, Core Router, Event Bus.

Vista técnica de alto nivel: API Gateway, IAM, BFF, microservicios BIAN, Core Router y backends.



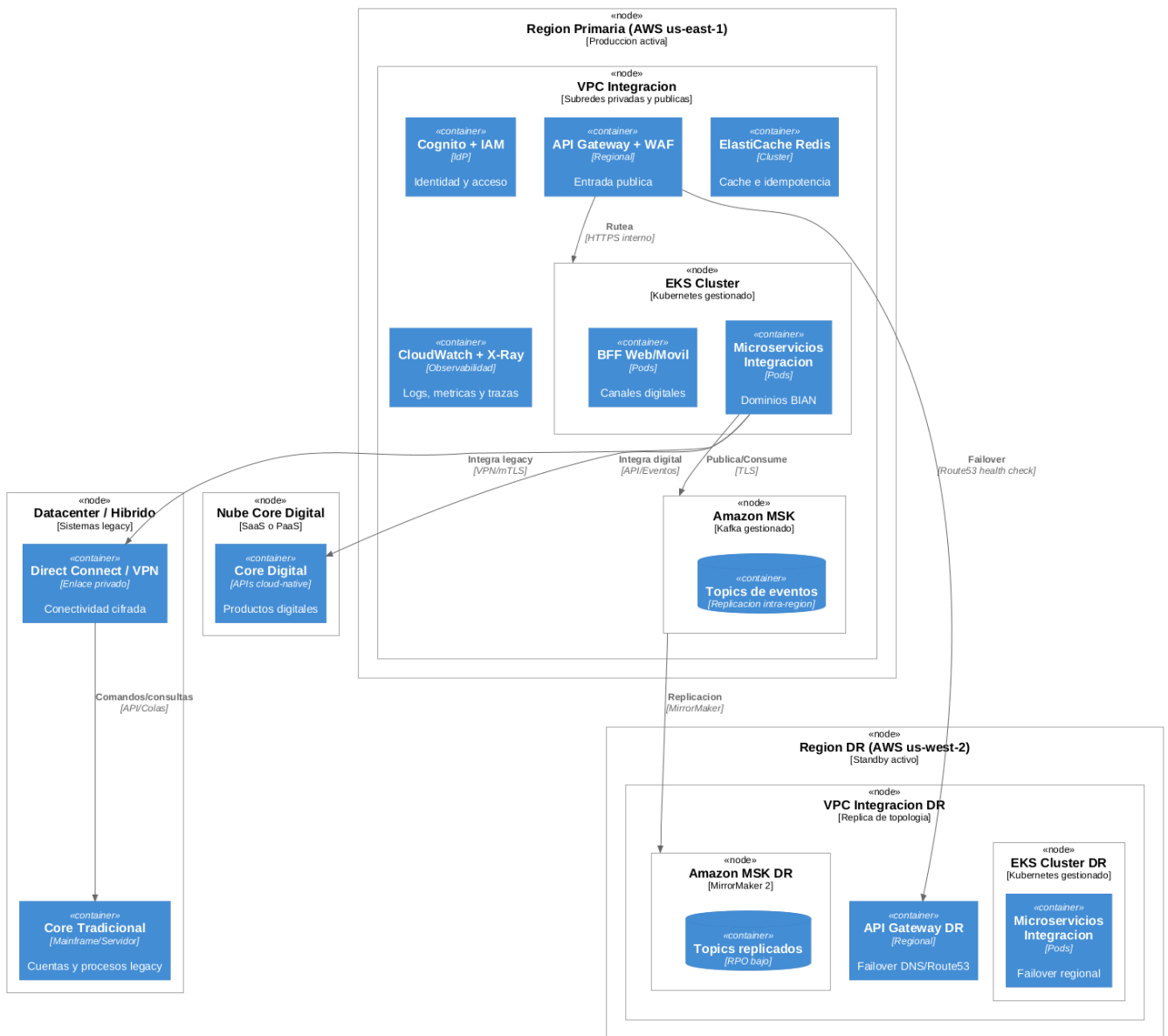
C4 Nivel 3 - Componentes

Policy Enforcement, Core Router, adapters, saga, eventos.



C4 Despliegue en Nube AWS

EKS, MSK, regiones primaria/DR, Direct Connect a legacy.



14. Conclusiones

- Este documento sustenta cada decisión de arquitectura con contexto, alternativas evaluadas y trazabilidad a componentes C4.
- La propuesta en nube AWS detalla topología de red, mapeo servicio a servicio, GitOps, capacidad y DR con métricas RTO/RPO concretas aplicadas a cada componente.
- La sección 6 documenta en profundidad seguridad, cumplimiento normativo y LOPD con marco de elementos, tablas de trazabilidad a componentes C4 y ADR-016 de privacy by design.
- La sección 8 documenta la estrategia de APIs internas/externas y mensajería bajo estándares de industria (OpenAPI, Avro, CloudEvents) con matriz sync vs async y ADR-019.
- La gestión de identidad y acceso (sección 5) complementa la seguridad con flujos por actor, autorización en tres capas y ciclo de vida de credenciales.
- La sección 9 define el modelo de gobierno de APIs y microservicios con RACI, ciclo de vida, estándares EKS, gobierno de eventos, KPIs y ADR-020.
- La sección 10 define el plan de migración gradual (requisito 9) con 5 fases, controles transversales, orden por dominio BIAN, comité go/no-go, procedimiento de rollback y ADR-021.
- La sección 11 define el modelo de observabilidad y trazabilidad (requisito 10) con tres pilares (métricas, logs, trazas), propagación transaction-id, SLO/SLI, Audit Trail, dashboards y ADR-022.
- La estrategia multicore con Core Router y patrón Strangler Fig permite migración gradual sin interrumpir operación del core tradicional.
- Los diagramas C4 (Contexto, Contenedores, Componentes y Despliegue) son coherentes con las secciones técnicas de este documento.